INF563 · VÉRIFICATION ET VALIDATION DE LOGICIEL · MASTER 2 SIGL

Solveur de l'équation du second degré : réflexion, pairwise, JaCoCo et PIT

Complément au document « Fonctionnement détaillé du projet » : ce qui a changé depuis la séance 3, le chargement de l'implémentation par **réflexion**, la stratégie **pairwise** sur les neuf catégories de réels, la **couverture de code** avec JaCoCo, le **test de mutation** avec PIT, et les réponses aux questions de l'enseignant sur les chapitres IV et V.

158	**81**	**100 %**	**38/39**
tests verts avec `mvnw verify`	cas pairwise au lieu de $9^3 = 729$	des branches couvertes (JaCoCo, 16/16)	mutants tués par PIT (97 %, 1 équivalent)

Enseignant : Evaris Fomekong · Niveau visé : chapitres IV (boîte noire, pairwise) et V (boîte blanche, couverture, mutation)
Projet : **Projet_test_polynome_chapitre3** · Java 21 · Maven 3.9.11 · JUnit 5.13.4 · AssertJ 3.27.3 · JaCoCo 0.8.12 · PIT 1.15.3
Document de compréhension et de préparation à l'oral · 4 novembre 2026

Table des matières

Comment lire ce complément **3**

1. Ce qui a changé depuis le 2 novembre **4**
- 1.1 La consigne du devoir 4
- 1.2 Correspondance consigne → réalisation 4
- 1.3 Avant / après 4
- 1.4 Les chiffres clés 5

2. La réflexion Java : charger l'implémentation par son nom **5**
- 2.1 Le problème : un test qui dépend d'une classe concrète 5
- 2.2 Qu'est-ce que la réflexion ? 6
- 2.3 Le contrat : l'interface SolveurSecondDegre 6
- 2.4 ChargeurSolveur pas à pas 6
- 2.5 Brancher la suite sur le chargeur 8
- 2.6 Les trois classes de test concrètes 8
- 2.7 Le profil Maven « configurable » 8
- 2.8 Commandes et verdicts 9
- 2.9 Les 11 tests du chargeur (ChargeurSolveurTest) 9
- 2.10 Le scénario « tester le code d'un camarade » 10

3. La stratégie pairwise sur les 9 catégories de réels **10**
- 3.1 Des classes d'équivalence aux 9 catégories 10
- 3.2 Each-choice, pairwise, exhaustif 11
- 3.3 Construire les 81 cas : un carré latin 11
- 3.4 Tester les données de test : PairwiseTest 12
- 3.5 Du générateur au test paramétré : @MethodSource 12
- 3.6 L'oracle des cas générés : ReferenceExacte 13
- 3.7 Les classes invalides : une seule par cas (diapo 90) 13
- 3.8 Résultats : pairwise contre exhaustif 14

4. Le défaut trouvé dans la version stable de la diapo 54 ..... **14**
- 4.1 Les quatre cas 14
- 4.2 La cause, pas à pas sur le cas 29 15
- 4.3 La correction, ligne par ligne 15
- 4.4 La preuve que la correction est juste 16

5. La couverture de code avec JaCoCo **16**
- 5.1 Comment JaCoCo mesure 16
- 5.2 La configuration 16
- 5.3 Les résultats 17
- 5.4 L'expérience : sans le pairwise ni les classes invalides 17
- 5.5 Graphe de flot de contrôle et complexité cyclomatique 18
- 5.6 Une base de 9 chemins indépendants 19
- 5.7 100 % de couverture ne prouve rien 19

6. Le test de mutation avec PIT **19**
- 6.1 Le principe 19
- 6.2 Les opérateurs de mutation appliqués 19
- 6.3 La configuration 20
- 6.4 Le problème de version (et comment on l'a trouvé) 21
- 6.5 Premier passage : 37 tués sur 41 21
- 6.6 Les quatre survivants et leur traitement 21
- 6.7 Résultat final : 38 tués sur 39 22
- 6.8 Les leçons des survivants 22
- 6.9 Couverture et mutation : complémentaires 22

7. Exécuter, lire les rapports, dépanner **23**
- 7.1 Les commandes (terminal VS Code : Ctrl + ù) 23
- 7.2 Dans VS Code, pas à pas 23
- 7.3 Dépannage 23

8. Questions probables de l'enseignant et réponses **24**
- 8.1 La réflexion 24
- 8.2 Le pairwise et l'oracle 24
- 8.3 La couverture (JaCoCo) 25
- 8.4 La mutation (PIT) 26
- 8.5 Questions d'ouverture 26

9. Mise à jour des chiffres de l'ancien document **26**
- 9.1 Les chiffres 26
- 9.2 Les sections à relire ici 27
- 9.3 « Détecte / ne détecte pas » : la version naïve, bloc par bloc 27

Annexes : le code source actuel **28**

Comment lire ce complément

> Ce document **complète** le premier (« Fonctionnement détaillé du projet », 56 pages, chapitres I à III). Il ne répète ni la théorie des flottants, ni le modèle formel d'un test, ni le rôle des annotations JUnit : il explique **tout ce qui a été ajouté** pour le devoir des chapitres IV et V, et ce qui a changé dans l'existant. Tous les chiffres cités viennent d'exécutions réelles du 4 novembre 2026 (Java 21, Maven 3.9.11).

Parcours conseillés

Si vous avez…	Lisez dans cet ordre
30 minutes avant l'oral	Partie 1 (ce qui a changé), les encadrés « À retenir », puis la partie 8 (questions et réponses)
Une soirée	Parties 1, 2, 3, 5 et 6, puis la partie 8
Envie de tout maîtriser	Tout, dans l'ordre, annexes comprises, avec VS Code ouvert pour relancer chaque commande citée

Codes visuels (les mêmes que dans le premier document)

> **À retenir**
> L'idée essentielle d'une section, à savoir dire en une phrase.

> **Si l'enseignant demande…**
> Une question probable et la réponse attendue, formulée simplement.

> **Piège**
> Une erreur classique (de code ou de raisonnement) et comment l'éviter.

> **Pour aller plus loin**
> Un détail technique utile si la discussion devient pointue.

Notations nouvelles

Notation	Signification
V[i]	La i-ème des 9 catégories de réels (i = 0 à 8, dans l'ordre de l'enum `CategorieReel`)
L81(9^3)	Tableau orthogonal de 81 lignes, 3 paramètres à 9 valeurs, de force 2 (pairwise)
MIN_VALUE, **MIN_NORMAL**	Plus petit double positif (≈ $4{,}9\cdot10^{-324}$, sous-normal) et plus petit double normal (≈ $2{,}2\cdot10^{-308}$)
$m\cdot2^e$	Écriture d'un double en mantisse m ∈ [1, 2[et exposant e : `Math.getExponent(x)`, `Math.scalb(x, k)` = $x\cdot2^k$
GFC, **V(G)**	Graphe de flot de contrôle ; complexité cyclomatique V(G) = E − N + 2 = nombre de décisions + 1
E, **N** (graphe)	Nombre d'arcs et de nœuds du GFC (à ne pas confondre avec la mesure d'erreur N du chapitre III)
Score de mutation	Mutants tués ÷ (mutants générés − mutants équivalents) (diapo 135)
M1 à M4	Les quatre mutants survivants du premier passage de PIT (partie 6)

1. Ce qui a changé depuis le 2 novembre

> À la séance 3, le projet testait `Solveur.racines(a, b, c)` avec 84 tests. Le devoir des chapitres IV et V ajoute quatre exigences : pairwise, réflexion, JaCoCo, PIT. Cette partie résume les changements ; les parties suivantes les expliquent une à une.

1.1 La consigne du devoir

> *« Le devoir consiste à reprendre le TP de résolution d'une équation du second degré en intégrant les notions abordées lors de la troisième séance du cours. Il faut utiliser JUnit comme framework de test, appliquer la stratégie Pairwise pour générer les combinaisons de tests à partir de nos neuf catégories de valeurs, et exploiter les tests paramétrés avec les annotations telles que @ParameterizedTest et @MethodSource. Le projet devra également intégrer la programmation par réflexion (Java Reflection API) afin de permettre le chargement et l'instanciation dynamique de l'implémentation à tester, sans dépendre directement de sa classe concrète. Après avoir fait ceci, il faut configurer le projet sous la forme d'un véritable projet Java avec Maven, intégrer JaCoCo afin de générer et d'analyser son rapport de couverture de code, dans le but d'évaluer le niveau de couverture des tests sur l'implémentation de l'équation du second degré, et appliquer une bibliothèque de mutation en utilisant PIT (diapo 138). »*

1.2 Correspondance consigne → réalisation

Exigence	Où c'est fait	Preuve
JUnit comme framework de test	JUnit 5.13.4 (Jupiter) pour toutes les classes de test ; AssertJ pour les comparaisons relatives	`mvnw test` : 158 tests
Pairwise sur les 9 catégories	`CategorieReel` (9 catégories, diapo 93), `Pairwise.tableauOrthogonal()` (81 cas), test `pairwise` ; propriété vérifiée par `PairwiseTest`	partie 3
@ParameterizedTest et @MethodSource	`pairwise` reçoit ses 81 cas par `@MethodSource("casPairwise")` ; 10 autres `@ParameterizedTest` (`@CsvSource`, `@ValueSource`, `@NullAndEmptySource`)	§ 3.5
Réflexion : chargement et instanciation dynamiques	Interface `SolveurSecondDegre` ; `ChargeurSolveur` (`Class.forName` → `getConstructor().newInstance()`) ; les tests ne donnent qu'un nom de classe	partie 2
Véritable projet Maven	`pom.xml`, Maven Wrapper (Maven 3.9.11), profils `defaut` / `naif` / `configurable`	§ 2.7
JaCoCo : rapport de couverture	Plugin 0.8.12 (diapo 130), `mvnw verify`, rapport `target/site/jacoco` (copie dans `doc/rapports/jacoco`), seuil 100 % des branches	partie 5
PIT : test de mutation (diapo 138)	Plugin `pitest-maven` 1.15.3 + `pitest-junit5-plugin` 1.2.3, rapport `target/pit-reports` (copie dans `doc/rapports/pit`), seuil 85 %	partie 6

1.3 Avant / après

Élément	Au 2 novembre (séance 3)	Au 4 novembre (chapitres IV et V)
Code de production	`Solveur.java` : classe `final`, constructeur privé, méthodes `static` `naif()` et `stable()`	Interface `SolveurSecondDegre` + classes publiques `SolveurNaif` et `SolveurStable` (constructeur public sans argument)
Système sous test g	Méthode `abstract` redéfinie par chaque sous-classe : `return Solveur.racines(a, b, c);`	Méthode `final` qui appelle une instance chargée **par réflexion** à partir d'un nom de classe
Classes de test	`SolveurStableTest`, `SolveurNaifTest`	+ `SolveurConfigurableTest`, `ChargeurSolveurTest`, `PairwiseTest` ; outils `ChargeurSolveur`, `CategorieReel`, `Pairwise`, `ReferenceExacte` ; pièce à conviction `SolveurStableDiapo54`

Élément	Au 2 novembre (séance 3)	Au 4 novembre (chapitres IV et V)
Bloc combinatoire	`combinaisons` : 3 × 4 × 3 = 36 cas, valeurs choisies à la main (produit cartésien)	`pairwise` : 81 cas sur les 9 catégories de réels (au lieu de $9^3 = 729$)
Contrat	a = 0 → `IllegalArgumentException`	+ coefficient NaN, +∞ ou −∞ → `IllegalArgumentException` (règle validée avant l'étape 2)
Version stable	Code de la diapo 54	**Corrigée** : le pairwise y a trouvé un défaut de sous-dépassement (partie 4)
Tests par classe de suite	84	142
`mvnw test`	84 tests	158 tests (142 + 11 + 5)
Échecs de la version naïve	18	28
Mesure des tests	aucune	JaCoCo (couverture) et PIT (mutation)

1.4 Les chiffres clés

Élément	Valeur	Commentaire
Fichiers Java	15	4 dans `src/main`, 11 dans `src/test`
Méthodes de test de `SuiteOraclesSolveur`	13	1 `@Test`, 11 `@ParameterizedTest`, 1 `@TestFactory`
Tests exécutés par classe de suite	142	1 + 121 cas paramétrés + 20 tests dynamiques (dont 81 pairwise)
`mvnw test` / `mvnw verify`	158	142 (stable) + 11 (chargeur) + 5 (générateur pairwise), 0 échec
Échecs voulus de la version naïve	28	12 sur les oracles du chapitre III, 15 en pairwise, 1 sur les cas ajoutés après PIT
Échecs de la version de la diapo 54	5	4 en pairwise + 1 cas ajouté après PIT
Couverture JaCoCo de `SolveurStable`	100 %	instructions 182/182, branches 16/16, lignes 19/19
Mutation PIT de `SolveurStable`	38/39	97 % ; le survivant est un mutant équivalent, justifié : 38/38 = 100 % hors équivalent

> **À retenir**
> Même suite d'oracles qu'à la séance 3, mais : l'implémentation est choisie **par son nom** (réflexion), les combinaisons viennent d'un **tableau pairwise** sur 9 catégories, et la qualité de la suite est **mesurée** deux fois : ce qu'elle exécute (JaCoCo, 100 % des branches) et ce qu'elle détecte (PIT, 38 mutants tués sur 39).

2. La réflexion Java : charger l'implémentation par son nom

> La consigne demande que les tests puissent charger et instancier l'implémentation **sans dépendre directement de sa classe concrète**. Les tests ne connaissent plus qu'une interface et une chaîne de caractères.

2.1 Le problème : un test qui dépend d'une classe concrète

À la séance 3, `SolveurStableTest` redéfinissait `g` par `return Solveur.racines(a, b, c);`. Le test était donc **compilé contre** la classe `Solveur` : pour tester une autre implémentation (celle d'un camarade, une version corrigée), il fallait écrire une nouvelle classe Java et recompiler. La dépendance est « directe » : si `Solveur` disparaît, les tests ne compilent plus.

Avec la réflexion, le nom de l'implémentation devient une **donnée** : une simple chaîne, comme `"inf563.solveur.SolveurStable"`, qu'on peut changer au lancement, sans toucher au code des tests.

2.2 Qu'est-ce que la réflexion ?

La **réflexion** (Java Reflection API, paquetages `java.lang` et `java.lang.reflect`) est la capacité d'un programme à manipuler **ses propres classes comme des objets** pendant qu'il s'exécute : retrouver une classe par son nom, connaître ses interfaces, ses constructeurs, ses méthodes, puis créer des objets ou appeler des méthodes sans les avoir nommés dans le code source.

Élément de l'API	Rôle dans le projet
`Class<?>`	Objet qui décrit une classe chargée en mémoire (son nom, ses interfaces, ses constructeurs…)
`Class.forName(nom)`	Demande au chargeur de classes (ClassLoader) de trouver, charger et initialiser la classe de ce nom complet
`A.class.isAssignableFrom(c)`	Vrai si un objet de la classe `c` peut être vu comme un `A` (ici : implémente l'interface)
`c.isInterface()`, `Modifier.isAbstract(c.getModifiers())`	Détecte ce qui ne peut pas être instancié
`c.asSubclass(A.class)`	Renvoie la même classe, typée `Class<? extends A>` : plus besoin de transtypage
`c.getConstructor()`	Le constructeur **public sans argument** (objet `Constructor`)
`constructeur.newInstance()`	Appelle ce constructeur : c'est le `new` de la réflexion

> **Pour aller plus loin : JUnit utilise déjà la réflexion**
>
> Le premier document (partie 5) l'expliquait : JUnit découvre les méthodes annotées `@Test` ou `@ParameterizedTest` par réflexion, crée l'instance de la classe de test par son constructeur et invoque les méthodes avec `Method.invoke`. Notre `ChargeurSolveur` fait la même chose, à plus petite échelle, pour le système sous test.

2.3 Le contrat : l'interface SolveurSecondDegre

Pour appeler `racines(a, b, c)` sur un objet dont on ne connaît pas la classe, il faut un **type connu à la compilation** : c'est le rôle de l'interface. Elle porte le prototype et les règles du contrat (exceptions, tableau vide, racine double, ordre croissant). Toute implémentation testable doit être une **classe publique**, implémenter cette interface et offrir un **constructeur public sans argument**.

`src/main/java/inf563/solveur/SolveurSecondDegre.java`

```java
package inf563.solveur;

/**
 * Fil rouge A : contrat d'un solveur de l'équation du second degré a·x² + b·x + c = 0.
 *
 * <p>Contrat commun à toutes les implémentations :
 * <ul>
 *   <li>a, b ou c vaut NaN, +∞ ou −∞ : IllegalArgumentException (coefficient non fini) ;</li>
 *   <li>a = 0 : IllegalArgumentException (ce n'est plus une équation du second degré) ;</li>
 *   <li>Δ &lt; 0 : tableau vide (pas de racine réelle) ;</li>
 *   <li>Δ = 0 : une racine double ;</li>
 *   <li>Δ &gt; 0 : deux racines triées dans l'ordre croissant.</li>
 * </ul>
 *
 * <p>Les tests ne dépendent que de cette interface : l'implémentation à tester est chargée
 * par réflexion à partir de son nom (voir ChargeurSolveur dans src/test). Une implémentation
 * doit donc être une classe publique avec un constructeur public sans argument.
 */
public interface SolveurSecondDegre {

    /** Racines réelles triées de a·x² + b·x + c = 0. */
    double[] racines(double a, double b, double c);
}
```

2.4 ChargeurSolveur pas à pas

`ChargeurSolveur` (dans `src/test`) contient une seule méthode, `charger(nomClasse)`. Chaque étape peut échouer ; chaque échec devient une `IllegalArgumentException` au message explicite, pour que le verdict ERROR dise immédiatement ce qui ne va pas.

#	Étape	Échec possible	Message de l'IllegalArgumentException
0	Nom vide ou absent	`null`, `""`, `" "`	aucun nom de classe fourni (propriété -Dsolveur.impl ?)
1	Chargement : `Class.forName(nom.strip())`	`ClassNotFoundException`	classe introuvable : …
2	Contrat : `isAssignableFrom`	la classe n'implémente pas l'interface	… n'implémente pas inf563.solveur.SolveurSecondDegre
3	Instanciable ? `isInterface`, `isAbstract`	interface ou classe abstraite	… est une interface ou une classe abstraite
4	Instanciation : `asSubclass(…).getConstructor().newInstance()`	`NoSuchMethodException`	… n'a pas de constructeur public sans argument
		`InvocationTargetException`	le constructeur de … a levé une exception (cause = la vraie exception)
		`ReflectiveOperationException` (autres)	impossible d'instancier …

src/test/java/inf563/solveur/ChargeurSolveur.java, lignes 26 à 52

```java
static SolveurSecondDegre charger(String nomClasse) {
    if (nomClasse == null || nomClasse.isBlank()) {
        throw new IllegalArgumentException("aucun nom de classe fourni (propriété -D" + PROPRIETE + " ?)");
    }
    // 1. Chargement : le nom (texte) devient un objet Class, sans dépendance à la compilation
    Class<?> classe;
    try {
        classe = Class.forName(nomClasse.strip());
    } catch (ClassNotFoundException e) {
        throw new IllegalArgumentException("classe introuvable : " + nomClasse, e);
    }
    // 2. Vérification du contrat : la classe doit implémenter l'interface et être instanciable
    if (!SolveurSecondDegre.class.isAssignableFrom(classe)) {
        throw new IllegalArgumentException(classe.getName() + " n'implémente pas " + SolveurSecondDegre.class.getName());
    }
    if (classe.isInterface() || Modifier.isAbstract(classe.getModifiers())) {
        throw new IllegalArgumentException(classe.getName() + " est une interface ou une classe abstraite");
    }
    // 3. Instanciation par le constructeur public sans argument
    try {
        return classe.asSubclass(SolveurSecondDegre.class).getConstructor().newInstance();
    } catch (NoSuchMethodException e) {
        throw new IllegalArgumentException(classe.getName() + " n'a pas de constructeur public sans argument", e);
    } catch (InvocationTargetException e) {
        throw new IllegalArgumentException("le constructeur de " + classe.getName() + " a levé une exception",
e.getCause());
    } catch (ReflectiveOperationException e) {     // InstantiationException, IllegalAccessException
        throw new IllegalArgumentException("impossible d'instancier " + classe.getName(), e);
```

Piège : getConstructor() ou getDeclaredConstructor() ?

`getConstructor()` ne trouve que les constructeurs **publics**. `getDeclaredConstructor()` trouverait aussi un constructeur privé, mais il faudrait forcer l'accès avec `setAccessible(true)`, c'est-à-dire contourner l'encapsulation de la classe testée. On a préféré un contrat explicite : « classe publique, constructeur public sans argument ».

Pour aller plus loin : InvocationTargetException

Si le constructeur de la classe chargée lève lui-même une exception, `newInstance()` ne la laisse pas passer telle quelle : il l'enveloppe dans une `InvocationTargetException`. On récupère la vraie cause avec `e.getCause()` et on la joint au message.

2.5 Brancher la suite sur le chargeur

La suite abstraite reçoit désormais un **nom** par son constructeur, charge l'implémentation et la range dans un champ typé par l'interface. `g` n'est plus abstraite : elle est `final` et délègue à cette instance.

src/test/java/inf563/solveur/SuiteOraclesSolveur.java, lignes 41 à 51

```
/** L'implémentation sous test, vue uniquement à travers l'interface. */
private final SolveurSecondDegre solveur;

/** JUnit crée une instance par test : chaque test reçoit une instance neuve, chargée par réflexion. */
SuiteOraclesSolveur(String nomImplementation) {
    solveur = ChargeurSolveur.charger(nomImplementation);
}

/** Le système sous test g : renvoie les racines triées de a·x² + b·x + c = 0. */
final double[] g(double a, double b, double c) {
    return solveur.racines(a, b, c);
```

JUnit crée une **instance neuve de la classe de test pour chaque test** (cycle de vie « par méthode ») : chaque test reçoit donc sa propre instance du solveur, chargée par réflexion. Ce n'est pas coûteux : `Class.forName` ne charge la classe qu'une fois (le ClassLoader la garde en mémoire) ; seul `newInstance()` est répété.

2.6 Les trois classes de test concrètes

Classe	Nom passé au chargeur	Filtre	Résultat
`SolveurStableTest`	`"inf563.solveur.SolveurStable"`	aucun tag : lancée par `mvnw test`	**142 verts**
`SolveurNaifTest`	`"inf563.solveur.SolveurNaif"`	`@Tag("naif")` : `-Pnaif`	**28 échecs voulus**
`SolveurConfigurableTest`	`System.getProperty("solveur.impl")`	`@Tag("configurable")` + `@EnabledIfSystemProperty`	selon la classe choisie

src/test/java/inf563/solveur/SolveurConfigurableTest.java, lignes 14 à 20

```
@Tag("configurable")
@EnabledIfSystemProperty(named = ChargeurSolveur.PROPRIETE, matches = ".+")
@DisplayName("Solveur choisi au lancement (-Dsolveur.impl=...)")
class SolveurConfigurableTest extends SuiteOraclesSolveur {

    SolveurConfigurableTest() {
        super(System.getProperty(ChargeurSolveur.PROPRIETE));
```

`@EnabledIfSystemProperty(named = "solveur.impl", matches = ".+")` désactive la classe tant que la propriété n'est pas donnée : JUnit ne l'instancie même pas, et le Test Explorer de VS Code l'affiche « désactivée ».

2.7 Le profil Maven « configurable »

Trois profils règlent les tags que Surefire inclut ou exclut. Le profil `configurable` s'active **tout seul** dès que la propriété `solveur.impl` est donnée en ligne de commande ; le profil `defaut` (`activeByDefault`) est alors automatiquement désactivé, comme le prévoit Maven.

Profil	Activation	Tags exclus	Tags inclus
`defaut`	par défaut	`naif,configurable`	(tous les autres)
`naif`	`-Pnaif`	—	`naif`
`configurable`	propriété `-Dsolveur.impl=…`	—	`configurable`

pom.xml, lignes 159 à 190

```
    <profile>
        <id>defaut</id>
        <activation>
            <activeByDefault>true</activeByDefault>
        </activation>
```

```
        <properties>
            <groupes.exclus>naif,configurable</groupes.exclus>
            <groupes.inclus></groupes.inclus>
        </properties>
    </profile>
    <profile>
        <id>naif</id>
        <properties>
            <groupes.exclus></groupes.exclus>
            <groupes.inclus>naif</groupes.inclus>
        </properties>
    </profile>
    <!-- S'active tout seul dès que -Dsolveur.impl=... est donné : seule la suite
         SolveurConfigurableTest s'exécute, contre la classe nommée. -->
    <profile>
        <id>configurable</id>
        <activation>
            <property>
                <name>solveur.impl</name>
            </property>
        </activation>
        <properties>
            <groupes.exclus></groupes.exclus>
            <groupes.inclus>configurable</groupes.inclus>
        </properties>
    </profile>
</profiles>
```

2.8 Commandes et verdicts

Commande (PowerShell, terminal VS Code)	Résultat observé
`.\mvnw.cmd test`	**158 tests, 0 échec**
`.\mvnw.cmd test -Pnaif`	**142 tests, 28 échecs (voulu)**
`.\mvnw.cmd test "-Dsolveur.impl=inf563.solveur.SolveurStable"`	**142 tests, 0 échec**
`.\mvnw.cmd test "-Dsolveur.impl=inf563.solveur.SolveurNaif"`	**142 tests, 28 échecs**
`.\mvnw.cmd test "-Dsolveur.impl=inf563.solveur.SolveurStableDiapo54"`	**142 tests, 5 échecs**
`.\mvnw.cmd test "-Dsolveur.impl=inf563.solveur.SolveurCamarade"` (classe absente)	chaque test en **ERROR** : « classe introuvable : inf563.solveur.SolveurCamarade »

> **Piège : les guillemets sous PowerShell**
> Sans guillemets, PowerShell découpe l'argument `-Dsolveur.impl=inf563.solveur.X` au premier point et Maven reçoit des morceaux incompréhensibles. Entre guillemets, l'argument arrive intact. Sous `cmd.exe` ou Git Bash, les guillemets sont facultatifs.

Le verdict d'un nom faux est **ERROR** et non FAIL : ce n'est pas une assertion (un oracle) qui rejette une sortie, c'est une exception imprévue levée avant même l'appel du système sous test, pendant la construction de la classe de test. C'est exactement la distinction FAIL / ERROR du chapitre III.

2.9 Les 11 tests du chargeur (ChargeurSolveurTest)

Le chargeur est lui-même du code : il a ses propres tests, exécutés une seule fois (pas par implémentation).

Test	Cas	Données	Oracle
`chargeParLeNom`	2	`SolveurStable`, `SolveurNaif`	bonne classe ; `racines(1, -3, 2)` = [1, 2]
`instanceNeuve`	1	deux chargements de `SolveurStable`	`assertNotSame` : deux objets distincts

Test	Cas	Données	Oracle
`refuseClasseInutilisable`	5	`inf563.solveur.Inexistant`	« introuvable »
		`java.lang.String`	« implémente pas »
		`inf563.solveur.SolveurSecondDegre`	« interface ou une classe abstraite »
		`ChargeurSolveurTest$SansConstructeurVide`	« constructeur public sans argument »
		`ChargeurSolveurTest$ConstructeurQuiEchoue`	« a levé une exception »
`refuseNomAbsent`	3	`null` , `""` , `" "`	« aucun nom de classe »

Le `$` du nom `ChargeurSolveurTest$SansConstructeurVide` est le nom « binaire » d'une classe imbriquée : c'est sous ce nom que `Class.forName` la trouve.

2.10 Le scénario « tester le code d'un camarade »

1. Copier sa classe dans `src/main/java` (par exemple `inf563/solveur/SolveurCamarade.java`) ; elle doit déclarer `implements SolveurSecondDegre` et avoir un constructeur public sans argument.
2. Lancer `.\mvnw.cmd test "-Dsolveur.impl=inf563.solveur.SolveurCamarade"` : les 142 tests de la suite s'exécutent contre son code, **sans écrire ni recompiler une seule ligne de test**.

> **Si l'enseignant demande : « et si sa classe n'implémente pas votre interface ? »**
> Le chargeur la refuse avec un message clair. On pourrait aller plus loin en cherchant la méthode par son nom (`getMethod("racines", double.class, double.class, double.class)`) et en l'appelant avec `Method.invoke` , mais on perdrait le contrat vérifié à la compilation : on a gardé l'interface, plus sûre et plus lisible.

> **À retenir**
> Les tests ne contiennent plus aucun `new SolveurStable()` ni aucun import d'une classe concrète : seulement l'interface et un nom. `Class.forName` charge, `isAssignableFrom` vérifie le contrat, `getConstructor().newInstance()` instancie, et chaque échec donne un message explicite.

3. La stratégie pairwise sur les 9 catégories de réels

> Chapitre IV : on dérive les cas de test de la spécification (boîte noire) en partitionnant le domaine de chaque entrée, puis on **réduit la combinatoire** par le pairwise : 81 cas au lieu de 729.

3.1 Des classes d'équivalence aux 9 catégories

Une classe d'équivalence est un sous-ensemble du domaine où l'on suppose que le système se comporte de la même façon (diapo 88) : un représentant par classe suffit si l'hypothèse est vraie. Pour un `double` , la diapo 93 donne les « classes usuelles des réels ». En séparant négatif et positif, on obtient les **neuf catégories** du devoir, codées dans l'enum `CategorieReel` avec un représentant chacune.

#	Catégorie	Représentant	Piège visé (diapo 93)	a	b	c
1	négatif	`-3.7`	ordres de grandeur variés dans un même test	✓	✓	✓
2	positif	`2.5`	idem	✓	✓	✓
3	zéro signé	`-0.0`	`0.0 == -0.0` est vrai mais `1/-0.0 = −∞`	✗	✓	✓
4	très petit	`1e-300`	proche des sous-normaux (MIN_NORMAL ≈ $2{,}2 \cdot 10^{-308}$)	✓	✓	✓
5	très grand	`1e300`	`x * x` donne +∞	✓	✓	✓

#	Catégorie	Représentant	Piège visé (diapo 93)	a	b	c
6	valeur spéciale	`NaN`	`NaN == NaN` est faux ; ±∞ testés à part (§ 3.7)	✗	✗	✗
7	non représentable	`0.1`	`0.1 + 0.2 != 0.3`	✓	✓	✓
8	entier > 2^{53}	`9_007_199_254_740_993L`	devient 9 007 199 254 740 992 = 2^{53} en double	✓	✓	✓
9	valeur entière	`3.0`	troncature `(int) -3.99 = -3`	✓	✓	✓

✓ : classe valide pour ce coefficient ; ✗ : classe invalide (rejet attendu). Zéro n'est invalide que pour a (ce n'est plus une équation du second degré) ; NaN est invalide partout (règle ajoutée au contrat).

3.2 Each-choice, pairwise, exhaustif

Stratégie	Critère (diapo 109)	Cas pour 3 paramètres × 9 valeurs
Each-choice (1-wise)	chaque valeur de chaque paramètre apparaît au moins une fois	au moins 9
Pairwise (2-wise)	chaque **paire** de valeurs de deux paramètres apparaît au moins une fois	au moins 9 × 9 = **81**
Exhaustif	toutes les combinaisons (produit cartésien)	9^3 = 729

Le fondement est empirique (études du NIST, Kuhn et al., diapo 109) : la plupart des défaillances sont déclenchées par **un ou deux** paramètres. Couvrir toutes les paires attrape donc l'essentiel des défauts pour une fraction du coût.

3.3 Construire les 81 cas : un carré latin

On numérote les catégories V[0] à V[8] (ordre de l'enum). Le cas (i, j), pour i et j de 0 à 8, est :

$$\text{cas}(i, j) = (\ a = V[i]\ ,\ b = V[j]\ ,\ c = V[(i + j) \bmod 9]\ )$$

- **Paires (a, b)** : les 81 couples (i, j) sont tous parcourus.
- **Paires (a, c)** : à i fixé, quand j parcourt 0..8, (i + j) mod 9 prend les 9 valeurs une fois chacune.
- **Paires (b, c)** : à j fixé, même raisonnement quand i varie.

Chaque paire de chaque couple de paramètres apparaît donc **exactement une fois** : c'est un tableau orthogonal L81(9^3), construit comme les tableaux L4(2^3) et L9 des diapos 109 et 110. 81 est le **minimum** possible, puisque les seules paires (a, b) en exigent 81.

`src/test/java/inf563/solveur/Pairwise.java, lignes 27 à 36`

```java
static List<CategorieReel[]> tableauOrthogonal() {
    CategorieReel[] v = CategorieReel.values();
    int n = v.length;
    List<CategorieReel[]> cas = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cas.add(new CategorieReel[]{v[i], v[j], v[(i + j) % n]});
        }
    }
    return cas;
```

a \ b	1 nég.	2 pos.	3 zéro	4 petit	5 grand	6 NaN	7 0,1	8 2^{53}	9 entier
1 nég.	1 nég.	2 pos.	3 zéro	4 petit	5 grand	6 NaN	7 0,1	8 2^{53}	9 entier
2 pos.	10 pos.	11 zéro	12 petit	13 grand	14 NaN	15 0,1	16 2^{53}	17 entier	18 nég.
3 zéro	19 zéro	20 petit	21 grand	22 NaN	23 0,1	24 2^{53}	25 entier	26 nég.	27 pos.
4 petit	28 petit	**29** **grand**	30 NaN	31 0,1	32 2^{53}	33 entier	34 nég.	35 pos.	36 zéro
5 grand	37 grand	38 NaN	39 0,1	40 2^{53}	41 entier	42 nég.	43 pos.	**44** **zéro**	**45** **petit**
6 NaN	46 NaN	47 0,1	48 2^{53}	49 entier	50 nég.	51 pos.	52 zéro	53 petit	54 grand
7 0,1	55 0,1	56 2^{53}	57 entier	58 nég.	59 pos.	60 zéro	61 petit	62 grand	63 NaN
8 2^{53}	64 2^{53}	65 entier	66 nég.	67 pos.	68 zéro	69 petit	70 grand	71 NaN	72 0,1
9 entier	73 entier	74 nég.	75 pos.	**76** **zéro**	77 petit	78 grand	79 NaN	80 0,1	81 2^{53}

Les 81 cas : ligne = catégorie de a, colonne = catégorie de b, case = catégorie de c (petit nombre = numéro du cas JUnit). ⬜ classe invalide (31 cas, exception attendue) · ⬜ échec de la version naïve (15) · ⬜ échec de la version de la diapo 54 (4)

> **Si l'enseignant demande : « pourquoi pas PICT ? »**
>
> PICT (diapo 111) est l'outil à utiliser quand les paramètres sont nombreux, de tailles différentes ou soumis à des contraintes. Ici, trois paramètres de même taille 9 : le carré latin atteint l'optimum de 81 cas, de façon démontrable, en dix lignes de Java, sans outil externe ; PICT ne pourrait pas faire moins de 81.

3.4 Tester les données de test : PairwiseTest

Un générateur de cas est du code : il peut être faux. `PairwiseTest` vérifie la propriété annoncée (5 tests).

Test	Cas	Ce qui est vérifié
`taille`	1	9 catégories ; 81 cas pairwise ; 729 cas exhaustifs
`toutesLesPaires`	3	pour (a, b), (a, c) et (b, c) : 81 paires distinctes, chacune exactement une fois
`equilibre`	1	chaque catégorie apparaît 9 fois pour a, pour b et pour c (each-choice en découle)

3.5 Du générateur au test paramétré : @MethodSource

`src/test/java/inf563/solveur/SuiteOraclesSolveur.java, lignes 170 à 190`

```
static Stream<Arguments> casPairwise() {
    return Pairwise.tableauOrthogonal().stream().map(t -> Arguments.of(t[0], t[1], t[2]));
}

/**
 * Oracle de chaque cas généré :
 * <ul>
 *   <li>une classe invalide (coefficient NaN, ou a = 0) : IllegalArgumentException attendue ;</li>
 *   <li>sinon : RA et Tol calculés par ReferenceExacte (Δ exact, puis 40 chiffres), puis N ≤ Tol.</li>
 * </ul>
 */
@ParameterizedTest(name = "[{index}] a = {0} | b = {1} | c = {2}")
@MethodSource("casPairwise")
@DisplayName("Pairwise : 9 catégories de réels, 81 cas au lieu de 729")
void pairwise(CategorieReel ca, CategorieReel cb, CategorieReel cc) {
    double a = ca.valeur, b = cb.valeur, c = cc.valeur;
    if (!Double.isFinite(a) || !Double.isFinite(b) || !Double.isFinite(c) || a == 0) {
        assertThrows(IllegalArgumentException.class, () -> g(a, b, c));
        return;
    }
    verifierContreReference(a, b, c);
```

- `@MethodSource("casPairwise")` désigne une méthode **statique** de la classe (elle est appelée avant toute instance de test) qui renvoie un `Stream<Arguments>` : une ligne par cas.
- `Arguments.of(t[0], t[1], t[2])` transmet directement les trois constantes de l'enum : aucune conversion de texte, contrairement à `@CsvSource`.
- Le nom affiché utilise `toString()` de l'enum : `[29] a = très petit (1.0E-300) | b = positif (2.5) | c = très grand (1.0E300)`.
- Une classe invalide (NaN quelque part, ou a = ±0) n'attend qu'une exception ; sinon l'oracle compare à une référence exacte (§ 3.6).

3.6 L'oracle des cas générés : ReferenceExacte

Les 81 cas sont générés : impossible d'écrire leurs résultats attendus à la main comme dans `parReference`. `ReferenceExacte` les calcule pour **n'importe quels** coefficients finis avec a ≠ 0, selon le modèle DE / RA / Tol du chapitre III.

1. **Nombre de racines sûr** : $\Delta = b^2 - 4ac$ est calculé **exactement** en `BigDecimal` (`new BigDecimal(double)` donne la valeur binaire exacte du double, et les produits ne sont pas arrondis).
2. **RA** : racines par la formule stable, en `BigDecimal` à 40 chiffres (aucune élimination), puis arrondies au double le plus proche par `doubleValue()` : ±∞ si la vraie racine dépasse MAX_VALUE, 0 si elle est sous MIN_VALUE.
3. **Tol** = $10 \cdot \kappa \cdot \varepsilon$ pour chaque racine, avec $\kappa = (|a|r^2 + |b||r| + |c|) / (|r| \cdot \sqrt{\Delta})$, la formule du RAPPORT (§ 2) ; racine double ($\Delta = 0$) : κ est infini, Tol = $10 \cdot \sqrt{\varepsilon}$ comme pour le cas 8 du chapitre III ; racine nulle (c = 0) : $\kappa = 1$, vérifiée en absolu.
4. **Verdict** `accepte(y, RA, Tol)` : si RA = ±∞, on exige y = RA ; sinon $|y - RA| \le Tol \cdot \max(|RA|, MIN_NORMAL)$.

src/test/java/inf563/solveur/ReferenceExacte.java, lignes 27 à 46

```
static ReferenceExacte de(double a, double b, double c) {
    BigDecimal A = new BigDecimal(a), B = new BigDecimal(b), C = new BigDecimal(c);  // valeurs exactes des double
    BigDecimal delta = B.multiply(B).subtract(QUATRE.multiply(A).multiply(C));        // exact
    if (delta.signum() < 0) {
        return new ReferenceExacte(new double[0], new double[0]);
    }
    if (delta.signum() == 0) {                    // racine double -b/2a : κ infini, on ne garde que ≈ √ε
        double r = B.negate().divide(DEUX.multiply(A), MC).doubleValue();
        return new ReferenceExacte(new double[]{r}, new double[]{10 * Math.sqrt(EPS)});
    }
    BigDecimal s = delta.sqrt(MC);
    BigDecimal q = (B.signum() >= 0 ? B.add(s) : B.subtract(s)).divide(DEUX.negate());  // -(b + signe(b)·√Δ)/2
    BigDecimal r1 = q.divide(A, MC), r2 = C.divide(q, MC);
    if (r1.compareTo(r2) > 0) {
        BigDecimal t = r1;
        r1 = r2;
        r2 = t;
    }
    return new ReferenceExacte(new double[]{r1.doubleValue(), r2.doubleValue()},
            new double[]{10 * kappa(A, B, C, r1, s) * EPS, 10 * kappa(A, B, C, r2, s) * EPS});
```

src/test/java/inf563/solveur/ReferenceExacte.java, lignes 68 à 72

```
static boolean accepte(double y, double ra, double tol) {
    if (Double.isInfinite(ra)) {
        return y == ra;
    }
    return Math.abs(y - ra) <= tol * Math.max(Math.abs(ra), Double.MIN_NORMAL);
```

> **Si l'enseignant demande : « ce max(|RA|, MIN_NORMAL), ce n'est pas le piège de la diapo 70 ? »**
>
> Non. La diapo 70 utilise `max(1, |RA|)` : pour toute racine inférieure à 1, la tolérance devient absolue et masque 25 % d'erreur sur 10^{-8}. Ici le plancher est MIN_NORMAL ≈ $2{,}2 \cdot 10^{-308}$, 10^{308} fois plus petit : il n'agit que dans le domaine des sous-normaux, où le double lui-même n'a plus de précision relative (diapo 93). Pour toute racine représentable normalement, le critère reste purement relatif.

3.7 Les classes invalides : une seule par cas (diapo 90)

Le pairwise combine parfois deux classes invalides, par exemple a = −0,0 et b = NaN : l'exception est levée pour la première raison et **masque la seconde** (diapo 90). Le test `pairwise` se contente alors de vérifier qu'une exception est levée. Pour respecter la règle «

une seule classe invalide par cas », le test `classeInvalide` reprend chaque valeur invalide **seule**, avec des valeurs valides (1, −3, 2) ailleurs : 11 cas.

`src/test/java/inf563/solveur/SuiteOraclesSolveur.java, lignes 223 à 231`

```
@ParameterizedTest(name = "[{index}] ({0}, {1}, {2}) → IllegalArgumentException")
@CsvSource({
        "NaN, -3, 2", "Infinity, -3, 2", "-Infinity, -3, 2",
        "1, NaN, 2", "1, Infinity, 2", "1, -Infinity, 2",
        "1, -3, NaN", "1, -3, Infinity", "1, -3, -Infinity",
        "0.0, -3, 2", "-0.0, -3, 2",
})
@DisplayName("Classes invalides : une seule par cas")
void classeInvalide(double a, double b, double c) {
```

3.8 Résultats : pairwise contre exhaustif

Implémentation	Pairwise (81 cas)	Exhaustif (729 cas)
`SolveurStable` (corrigée)	**0 échec**	**0 échec**
`SolveurStableDiapo54`	**4 échecs (4,9 %)**	**33 échecs (4,5 %)**
`SolveurNaif`	**15 échecs (18,5 %)**	**145 échecs (19,9 %)**

Le contrôle exhaustif a été fait une fois, hors de la suite, par un petit programme utilisant **le même oracle**. Le pairwise détecte les deux implémentations défectueuses avec **9 fois moins de cas**, dans la même proportion que l'exhaustif : les défauts sont bien déclenchés par des **paires** de coefficients (par exemple a très petit avec c très grand), ce qui est exactement l'hypothèse du pairwise.

Défaut de la version naïve (chapitre II)	Cas	Numéros et exemple
Débordement : $b^2 \to +\infty$	6	5, 32, 41, 59, 68, 77 ; cas 59 : y = −∞ pour RA = $-1{,}0\cdot10^{301}$
Absorption puis élimination totale : petite racine = 0,0	6	17, 28, 34, 35, 61, 80 ; cas 34 : y = 0,0 pour RA = 37
Élimination partielle	2	8 et 55 ; cas 8 : y = −0,81 pour RA = −1,0 (19 % d'erreur)
Sous-dépassement : $b^2 \to 0$, Δ = 0 à tort	1	76 : 1 racine au lieu de 2

> **À retenir**
>
> 9 catégories de réels (diapo 93) × 3 coefficients : 729 combinaisons ; un carré latin en garde 81 qui couvrent toutes les paires (vérifié par `PairwiseTest`). L'oracle des cas générés est une référence exacte en `BigDecimal` avec Tol = 10·κ·ε. Résultat : la naïve échoue sur 15 cas, la version de la diapo 54 sur 4.

4. Le défaut trouvé dans la version stable de la diapo 54

> Le premier lancement du test pairwise a fait échouer la version « stable » du cours sur 4 cas sur 81 : c'est le meilleur argument en faveur du pairwise, qui a trouvé un défaut que les 84 tests de la séance 3 ne voyaient pas.

4.1 Les quatre cas

Cas	(a, b, c)	Vraies racines	Diapo 54 renvoie
29	$(10^{-300}, 2{,}5, 10^{300})$	$-2\cdot10^{300}$ et $-5\cdot10^{299}$	**1 racine**
44	$(10^{300}, 2^{53}, -0{,}0)$	$-9\cdot10^{-285}$ et 0	**1 racine**
45	$(10^{300}, 3, 10^{-300})$	≈ $-2{,}6\cdot10^{-300}$ et $-3{,}8\cdot10^{-301}$	**1 racine**
76	$(3, 10^{-300}, -0{,}0)$	$-3{,}3\cdot10^{-301}$ et 0	**1 racine**

Ce sont des entrées **légales** (coefficients finis, a ≠ 0) dont les racines sont **représentables** en double : l'algorithme devait les trouver.

4.2 La cause, pas à pas sur le cas 29

src/test/java/inf563/solveur/SolveurStableDiapo54.java, lignes 21 à 27

```
// mise à l'échelle exacte par une puissance de 2
double m = Math.max(Math.abs(a), Math.max(Math.abs(b), Math.abs(c)));
int e = Math.getExponent(m);
a = Math.scalb(a, -e);
b = Math.scalb(b, -e);
c = Math.scalb(c, -e);
double d = b * b - 4 * a * c;
```

1. m = max(|a|, |b|, |c|) = 10^{300}, donc e = `getExponent(1e300)` = 996.
2. a' = $10^{-300}\cdot 2^{-996} \approx 1{,}5\cdot 10^{-600}$: bien en dessous de MIN_VALUE ≈ $4{,}9\cdot 10^{-324}$, donc a' = 0.
3. b' = $2{,}5\cdot 2^{-996} \approx 3{,}7\cdot 10^{-300}$, donc $b'^2 \approx 1{,}4\cdot 10^{-599} = 0$ en double.
4. d = $b'^2 - 4a'c' = 0 - 0 = 0$: la méthode répond « racine double » ($-b'/2a' = -\infty$), alors que le vrai Δ vaut 6,25 − 4 = 2,25 > 0 (le produit ac vaut environ 1).

La mise à l'échelle **commune** protège du débordement ($b^2 \to +\infty$), mais pas de son symétrique : le **sous-dépassement** (underflow) des petits coefficients, dès qu'ils sont beaucoup plus petits que le plus grand.

4.3 La correction, ligne par ligne

src/main/java/inf563/solveur/SolveurStable.java, lignes 20 à 42

```
public double[] racines(double a, double b, double c) {
    if (!Double.isFinite(a) || !Double.isFinite(b) || !Double.isFinite(c))
        throw new IllegalArgumentException("coefficient non fini (NaN ou infini)");
    if (a == 0) throw new IllegalArgumentException("a = 0");
    if (c == 0) {                                   // x·(a·x + b) = 0 : racines 0 et −b/a
        if (b == 0) return new double[]{0.0};       // Δ = 0 : racine double 0
        double r = -b / a;
        return new double[]{Math.min(r, 0.0), Math.max(r, 0.0)};
    }
    // x = m·2^e, mantisse m dans [1, 2[ : multiplier par une puissance de 2 est exact
    int ea = Math.getExponent(a), eb = Math.getExponent(b), ec = Math.getExponent(c);
    double ma = Math.scalb(a, -ea), mb = Math.scalb(b, -eb), mc = Math.scalb(c, -ec);
    // Δ = 2^e·d : e = exposant du plus grand des deux termes b² et 4ac, rendu pair pour la racine carrée
    // (b = 0 : getExponent(0) = −1023, donc 2·eb ≤ ea + ec et le max choisit bien 4ac)
    int e = Math.max(2 * eb, ea + ec);
    e += e & 1;                                     // pair supérieur (le pair inférieur marcherait aussi)
    double d = Math.scalb(mb * mb, 2 * eb - e) - Math.scalb(4 * ma * mc, ea + ec - e);
    if (d < 0) return new double[0];
    if (d == 0) return new double[]{-(b / a) / 2};
    double s = Math.scalb(Math.sqrt(d), e / 2);     // √Δ = √d · 2^(e/2)
    double q = -0.5 * (b + Math.copySign(s, b));
    double r1 = q / a, r2 = c / q;                  // Viète : r1 * r2 = c / a
    return new double[]{Math.min(r1, r2), Math.max(r1, r2)};
```

Lignes	Rôle
21 à 23	Contrat : coefficient non fini, puis a = 0
24 à 28	Cas c = 0 traité directement : x·(a·x + b) = 0 donne 0 et −b/a, sans calculer Δ (sinon b^2 pourrait passer sous MIN_VALUE : c'est le mutant M1 de la partie 6)
30 et 31	Chaque coefficient s'écrit $x = m\cdot 2^e$: `getExponent` et `scalb` sont exacts ; m ∈ [1, 2[(0 si b = 0)
34 et 35	e = exposant du plus grand des deux termes b^2 et 4ac, rendu pair pour pouvoir prendre la racine carrée de 2^e
36	$\Delta = 2^e\cdot d$: les deux termes sont mis à l'échelle **séparément** ; le plus grand vaut entre 0,5 et 8 environ, le plus petit ne peut devenir nul que s'il est réellement négligeable devant l'autre
37 et 38	Δ < 0 : aucune racine ; Δ = 0 : racine double −(b/a)/2
39	$\sqrt{\Delta} = \sqrt{d}\cdot 2^{e/2}$, exact pour la mise à l'échelle

Lignes	Rôle
40 et 41	Formule stable $q = -\frac{1}{2}(b + \text{signe}(b)\cdot\sqrt{\Delta})$, puis $r_1 = q/a$ et $r_2 = c/q$: **une seule division** chacune, donc $\pm\infty$ ou 0 seulement si la vraie racine sort du domaine des double

4.4 La preuve que la correction est juste

- Pairwise : 81 cas sur 81 ; contrôle exhaustif : **729 cas sur 729**.
- Les oracles du chapitre III passent toujours tous (référence, propriétés, métamorphiques, construction, différentiel).
- `Demo` affiche exactement les mêmes résultats qu'avant sur les 6 cas du TP de la séance 2.
- La version d'origine est gardée dans `src/test` sous le nom `SolveurStableDiapo54` : `.\mvnw.cmd test "-Dsolveur.impl=inf563.solveur.SolveurStableDiapo54"` donne **5 échecs** (les 4 cas pairwise + le cas (10^{300}, 10^{-300}, 0) ajouté après PIT). Étant une classe de test, elle n'est ni livrée ni mesurée par JaCoCo et PIT.

> **Si l'enseignant demande : « pourquoi avoir modifié le code du cours ? »**
> Parce que le TP de la séance 2 demande précisément de trouver des entrées qui mettent le solveur en défaut, puis de le corriger (diapo 50). Le pairwise a trouvé 4 entrées légales, à racines représentables, où la diapo 54 renvoie un mauvais nombre de racines : c'est un vrai défaut, pas une question de tolérance. L'original reste disponible pour le démontrer.

5. La couverture de code avec JaCoCo

> Chapitre V, boîte blanche : on regarde maintenant le code. La couverture répond à la question « **quel code les tests exécutent-ils ?** » (diapos 130 et 131).

5.1 Comment JaCoCo mesure

prepare-agent prépare l'option `-javaagent` pour Surefire → **tests (Surefire)** l'agent instrumente le bytecode au chargement des classes → **target/jacoco.exec** compteurs d'exécution, écrits à la fin des tests → **report** HTML, XML, CSV dans `target/site/jacoco` → **check** seuil : le build échoue en dessous (quality gate)

L'agent ne modifie pas les fichiers `.class` sur le disque : il ajoute des sondes au bytecode en mémoire, au moment où chaque classe est chargée. `report` et `check` sont rattachés à la phase `verify` : ils s'exécutent avec `.\mvnw.cmd verify`, pas avec `test`.

5.2 La configuration

`pom.xml : plugin JaCoCo`

```
<artifactId>jacoco-maven-plugin</artifactId>
<version>0.8.12</version>
<configuration>
    <!-- On mesure l'implémentation livrée (SolveurStable). Exclues :
         Demo (affichage, pas un calcul) et SolveurNaif (contre-exemple pédagogique,
         ses tests sont exclus du build par défaut). -->
    <excludes>
        <exclude>inf563/solveur/Demo.class</exclude>
        <exclude>inf563/solveur/SolveurNaif.class</exclude>
    </excludes>
</configuration>
<executions>
    <execution>
        <goals>
            <goal>prepare-agent</goal>
        </goals>
    </execution>
    <execution>
        <id>rapport</id>
        <phase>verify</phase>
        <goals>
```

```
            <goal>report</goal>
            <goal>check</goal>
        </goals>
        <configuration>
            <!-- Diapo 130 : 0.80 ; TP diapo 140 : 100 % des branches. Atteint
                 (16/16 branches de SolveurStable) : le seuil empêche de régresser. -->
            <rules>
                <rule>
                    <element>BUNDLE</element>
                    <limits>
                        <limit>
                            <counter>BRANCH</counter>
                            <value>COVEREDRATIO</value>
                            <minimum>1.00</minimum>
                        </limit>
                    </limits>
                </rule>
            </rules>
        </configuration>
    </execution>
</executions>
```

Choix	Justification
Version 0.8.12	Celle de la diapo 130 ; elle lit le bytecode de Java 21.
Exclusion de `Demo`	Un programme d'affichage, pas un calcul : il serait à 0 % et noierait la mesure.
Exclusion de `SolveurNaif`	Contre-exemple pédagogique, pas l'implémentation livrée ; sa suite (qui échoue exprès) est exclue du build par défaut, il apparaîtrait donc presque non couvert.
Seuil BRANCH ≥ 1.00	La diapo 130 montre 0.80 ; le TP de la diapo 140 demande 100 % des branches. Atteint, le seuil empêche toute régression.

5.3 Les résultats

Compteur (diapo 131)	SolveurStable	Ce qu'il mesure
Instructions	**182 / 182**	instructions de bytecode exécutées
Branches	**16 / 16**	issues des sauts conditionnels (if, `||`) : 8 décisions × 2
Lignes	**19 / 19**	lignes de source avec au moins une instruction exécutée
Complexité	**10 / 10**	V(G) : 9 pour `racines` + 1 pour le constructeur implicite
Méthodes	**2 / 2**	`racines` et le constructeur

Dans le rapport HTML, chaque ligne du source est colorée : **vert** = couverte, **jaune** = une branche manque (losange jaune dans la marge), **rouge** = jamais exécutée. Les lignes jaunes sont les plus instructives. L'interface `SolveurSecondDegre` ne contient pas de code exécutable : elle n'apparaît pas dans les compteurs.

5.4 L'expérience : sans le pairwise ni les classes invalides

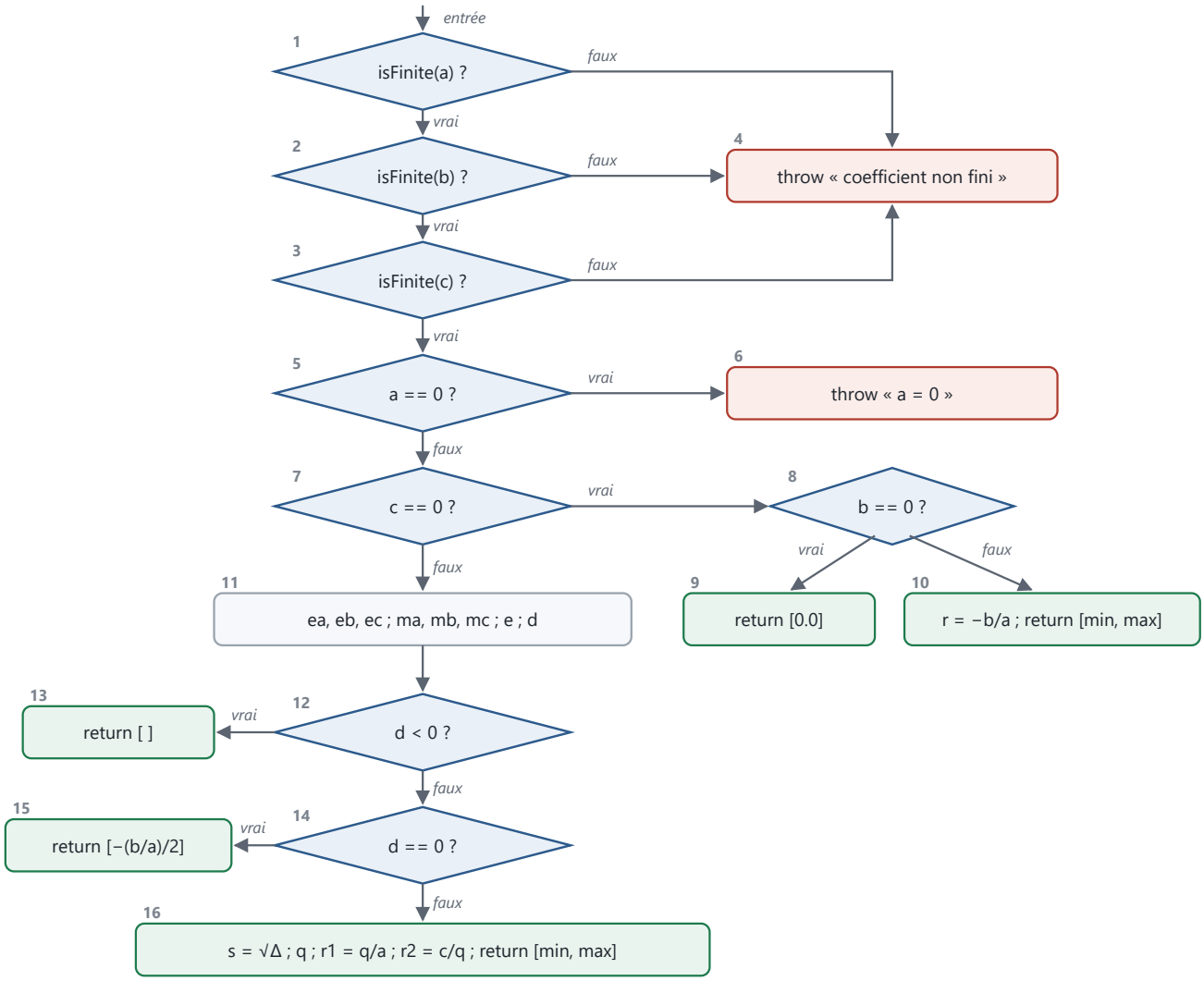
Pour vérifier que le seuil sert à quelque chose, on a relancé `verify` en ne gardant que les oracles du chapitre III (48 tests : `parReference`, `aNul`, `discriminantNegatif`, `residu`, `viete`, `echelle`, `oppose`, `construction`, `constructionGeneree`, `differentiel`), avec l'option `-Dtest=SolveurStableTest#parReference+…`.

Mesure	Oracles du chapitre III seuls	Suite complète
Branches	**12 / 18 (66 %)**	**18 / 18 (100 %)**
Lignes	**15 / 19**	**19 / 19**
Build	**FAILURE**	**SUCCESS**

Message obtenu : `Rule violated for bundle solveur-second-degre: branches covered ratio is 0.66, but expected minimum is 1.00`. Les branches manquantes sont les branches « vrai » des trois `isFinite` (ligne 21), le `throw` (ligne 22) et tout le

cas c = 0 (lignes 24 à 27) : exactement ce qu'apportent les tests `classeInvalide` et `pairwise`. Cette mesure a été faite **avant** la suppression du ternaire (partie 6), d'où 18 branches au lieu de 16.

5.5 Graphe de flot de contrôle et complexité cyclomatique



GFC de `SolveurStable.racines` : 8 décisions (losanges), 2 blocs de calcul ; nœuds terminaux en rouge (exception) et en vert (return). Chaque nœud terminal a un arc vers un nœud de sortie unique, non dessiné.

Calcul	Valeur
Nœuds N	16 dessinés + 1 nœud de sortie = **17**
Arcs E	17 arcs dessinés + 7 arcs des nœuds terminaux vers la sortie = **24**
V(G) = E − N + 2	24 − 17 + 2 = **9**
V(G) = décisions + 1	8 + 1 = **9** (3 `isFinite`, a == 0, c == 0, b == 0, d < 0, d == 0)
JaCoCo	complexité de `racines` = **9**

> **Pour aller plus loin : 9, 7 ou 10 ?**
> JaCoCo travaille sur le bytecode, où chaque condition de `a || b || c` est un saut séparé : il compte **3** décisions pour la ligne 21. En lisant le source et en comptant la condition composée comme une seule décision, on trouverait 6 décisions, donc V(G) = 7. On retient la convention de JaCoCo (conditions élémentaires). Avant la suppression du ternaire `(b == 0) ? … : …` (partie 6), il y avait 9 décisions : V(G) = 10.

5.6 Une base de 9 chemins indépendants

V(G) = 9 : il faut au moins 9 tests pour une base de chemins indépendants (diapo 122). Chacun est couvert par un test existant.

#	Chemin	Données	Test	Sortie
1	a non fini	(NaN, −3, 2)	`classeInvalide`	exception
2	b non fini	(1, NaN, 2)	`classeInvalide`	exception
3	c non fini	(1, −3, NaN)	`classeInvalide`	exception
4	a = 0	(0, 2, 1)	`aNul`	exception
5	c = 0 et b = 0	(−3,7 ; −0,0 ; −0,0)	`pairwise` [3]	[0]
6	c = 0 et b ≠ 0	(10^{300}, 2^{53}, −0,0)	`pairwise` [44]	2 racines
7	Δ < 0	(1, 1, 1)	`discriminantNegatif`	[ ]
8	Δ = 0	(1, −2, 1)	`parReference`	[1]
9	Δ > 0	(1, −3, 2)	`parReference`	[1, 2]

5.7 100 % de couverture ne prouve rien

La couverture mesure ce qui est **exécuté**, pas ce qui est **vérifié** (diapo 132) : un test qui appellerait `g` sur les 9 jeux de données ci-dessus **sans aucune assertion** atteindrait lui aussi 100 % des branches, et ne détecterait rien. Une couverture basse prouve qu'il manque des tests ; une couverture haute ne dit rien de leur qualité. D'où le test de mutation : à 100 % de branches, PIT a encore trouvé 4 mutants survivants (partie 6).

> **À retenir**
> JaCoCo instrumente le bytecode pendant les tests ; `mvnw verify` produit le rapport et applique le seuil. `SolveurStable` est couverte à 100 % (16/16 branches) ; sans le pairwise ni les classes invalides, on tombe à 66 % et le build échoue. V(G) = 9, couvert par une base de 9 chemins.

6. Le test de mutation avec PIT

> La mutation répond à l'autre question : « **quelles fautes les tests détectent-ils ?** ». Elle voit l'oracle, là où la couverture ne le voit pas (diapo 139).

6.1 Le principe

1. **Muter** : PIT crée des copies du bytecode de `SolveurStable`, chacune avec une petite faute (un **mutant**) : `/` remplacé par `*`, une condition remplacée par `false`…
2. **Exécuter** : il lance sur chaque mutant les tests qui couvrent la ligne modifiée.
3. **Tué ?** Au moins un test échoue : le mutant est **tué**. Sinon il **survit**.
4. **Analyser** chaque survivant : il manque un test… ou le mutant est **équivalent** (même comportement que l'original, aucun test ne peut le tuer).

Deux hypothèses fondent la méthode (DeMillo, Lipton, Sayward, 1978) : le **programmeur compétent** (les fautes réelles sont de petites erreurs) et l'**effet de couplage** (une suite qui détecte les fautes simples détecte la plupart des fautes complexes).

$$\text{score de mutation} = \text{mutants tués} \div (\text{mutants générés} - \text{mutants équivalents})$$

6.2 Les opérateurs de mutation appliqués

Opérateur (groupe DEFAULTS de PIT 1.15.3)	Exemple dans SolveurStable	Mutants
MATH	`b / a` → `b * a`, `e += …` → `e -= …`	20

Opérateur (groupe DEFAULTS de PIT 1.15.3)	Exemple dans SolveurStable	Mutants
REMOVE_CONDITIONALS_EQUAL_ELSE	`if (c == 0) → if (false)`	7
INVERT_NEGS	`-b → b`	5
NULL_RETURNS	`return new double[]{…} → return null`	5
CONDITIONALS_BOUNDARY	`d < 0 → d <= 0`	1
REMOVE_CONDITIONALS_ORDER_ELSE	`if (d < 0) → if (false)`	1
Total (passage final)		**39**

La diapo 134 cite aussi NEGATE_CONDITIONALS (`==` → `!=`). Dans le rapport de PIT 1.15.3, aucun mutant de ce type n'apparaît : les conditions y sont mutées par REMOVE_CONDITIONALS_*_ELSE (condition remplacée par `false`).

6.3 La configuration

`pom.xml : plugin PIT`

```xml
<artifactId>pitest-maven</artifactId>
<version>1.15.3</version>
<dependencies>
    <!-- Diapo 138 : 1.2.1. Celle-ci tire junit-platform-launcher 1.9.2, incompatible
         avec JUnit 5.13 (« OutputDirectoryProvider not available ») : 1.2.3 corrige. -->
    <dependency>
        <groupId>org.pitest</groupId>
        <artifactId>pitest-junit5-plugin</artifactId>
        <version>1.2.3</version>
    </dependency>
</dependencies>
<configuration>
    <!-- on mute l'implémentation livrée, comme pour JaCoCo -->
    <targetClasses>
        <param>inf563.solveur.SolveurStable</param>
    </targetClasses>
    <!-- PIT exige une suite VERTE avant de muter : on écarte la suite naïve
         (échecs voulus) et la suite configurable (sans objet ici) -->
    <excludedGroups>
        <param>naif</param>
        <param>configurable</param>
    </excludedGroups>
    <mutators>
        <mutator>DEFAULTS</mutator>
    </mutators>
    <mutationThreshold>85</mutationThreshold>
    <timestampedReports>false</timestampedReports>
    <outputFormats>
        <param>HTML</param>
        <param>XML</param>
    </outputFormats>
```

Paramètre	Pourquoi
`targetClasses`	On ne mute que `SolveurStable` , l'implémentation livrée (même périmètre que JaCoCo).
`excludedGroups`	PIT exécute d'abord la suite **sans mutation** pour savoir quels tests couvrent quelles lignes ; elle doit être **verte**, sinon tous les mutants seraient « tués » par des tests déjà rouges. On écarte donc les tags `naif` (échecs voulus) et `configurable` .
`mutators` = DEFAULTS	Les opérateurs par défaut, comme dans la diapo 138.
`mutationThreshold` = 85	Objectif du TP de la diapo 140 : le build échoue sous 85 %.
`timestampedReports` = false	Rapport toujours au même endroit : `target/pit-reports/index.html` .
`outputFormats` HTML + XML	HTML pour lire ; XML (`mutations.xml`) pour lister les survivants et les tests qui tuent.

Lancement, comme dans la diapo 138 : `.\mvnw.cmd test-compile org.pitest:pitest-maven:mutationCoverage` . PIT n'est pas rattaché à `verify` : il est beaucoup plus coûteux (une exécution de tests par mutant).

6.4 Le problème de version (et comment on l'a trouvé)

Avec les versions exactes de la diapo 138 (pitest-maven 1.15.3, pitest-junit5-plugin 1.2.1), le premier lancement a échoué : `Coverage generation minion exited abnormally! (UNKNOWN_ERROR)`. Relancé avec `-Dverbose=true`, le processus « minion » (la JVM séparée où PIT exécute les tests) affichait la vraie cause :

```
OutputDirectoryProvider not available; probably due to unaligned versions of the
junit-platform-engine and junit-platform-launcher jars on the classpath/module path.
```

`pitest-junit5-plugin` 1.2.1 tire `junit-platform-launcher` **1.9.2**, qui se retrouve sur le classpath avant la version 1.13.4 qui accompagne JUnit 5.13.4 : lanceur et moteur ne sont plus alignés. La version **1.2.3** du plugin (la dernière) corrige le problème ; `pitest-maven` reste en 1.15.3, comme dans la diapo. Le changement est commenté dans le `pom.xml`.

6.5 Premier passage : 37 tués sur 41

Résultat : `Generated 41 mutations Killed 37 (90%)`, couverture de lignes 19/19, 551 exécutions de tests (13,44 par mutant). Le seuil de 85 % était déjà atteint, mais le TP demande d'analyser **chaque** survivant (diapo 140, étape 4).

6.6 Les quatre survivants et leur traitement

#	Mutation (ligne du 1er passage)	Verdict	Traitement
M1	l. 24 : `if (c == 0)` → `if (false)`	**pas équivalent**	Test ajouté (10^{300}, 10^{-300}, 0) : **tué**
M2	l. 33 : `(b == 0) ? …` → `false`	**équivalent**	Ternaire **supprimé** (code inutile) : le mutant disparaît
M3	l. 34 : `e += e & 1` → `e -= e & 1`	**équivalent**	Gardé, **justifié par écrit** (ci-dessous, commentaire du code, README)
M4	l. 37 : `-(b / a) / 2` → `-(b * a) / 2`	**pas équivalent**	Test ajouté (4, −4, 1) : **tué**

M1 : il manquait une valeur limite

Sans le cas particulier c = 0, le chemin général traite c = 0… sauf quand b^2 passe sous MIN_VALUE après mise à l'échelle. Pour (10^{300}, 10^{-300}, 0) : ea = 996, eb = −997, ec = `getExponent(0)` = −1023 ; e = max(−1994, −27) = −27, rendu pair : −26 ; d = scalb(mb^2, −1994 + 26 = −1968) = 0. Le mutant répond « racine double » alors que $\Delta = b^2 > 0$: il y a deux racines, 0 et -10^{-600} (arrondie à −0,0). Aucun test ne contenait ce cas.

M2 : du code inutile

Si b = 0, `getExponent(0)` vaut −1023, donc 2·eb = −2046, alors que ea + ec ≥ −2046 pour tous coefficients finis : `max(2·eb, ea + ec)` donne déjà ea + ec. Le ternaire ne changeait rien : aucun test ne pouvait le tuer. Plutôt que de justifier un mutant équivalent, on a supprimé la condition : le code est plus simple, V(G) passe de 10 à 9 et le nombre de branches de 18 à 16.

M3 : un mutant équivalent, justifié

Quand e est impair, l'original l'arrondit au pair supérieur (e + 1), le mutant au pair inférieur (e − 1). Dans le mutant, les deux termes de d sont décalés de 2 exposants de plus : $d_{mutant} = 4 \cdot d$ exactement (multiplier par une puissance de 2 est exact ; le grand terme reste normal). Puis $\sqrt{(4 \cdot d)} = 2 \cdot \sqrt{d}$ **exactement** (la racine IEEE est correctement arrondie et le facteur 2 est exact), et le facteur 2 est compensé par l'exposant (e − 1)/2 = (e + 1)/2 − 1 : $\sqrt{\Delta}$ est identique **bit pour bit**. Seul cas douteux : si le petit terme est sous-normal, son arrondi peut différer, mais il est alors inférieur à la moitié d'un ulp du grand terme (≥ 0,5) et disparaît dans la soustraction, dans les deux versions. Aucune entrée ne distingue le mutant : il est **équivalent** (comme le M1 de la diapo 137).

M4 : une donnée de test trop particulière

Le seul test de racine double était (1, −2, 1) : avec a = 1, diviser ou multiplier par a donne le même résultat, 1 étant l'élément neutre. Le cas (4, −4, 1) a Δ = 16 − 16 = 0 et pour racine double 0,5 ; le mutant renvoie −(−4·4)/2 = 8 : il est tué.

src/test/java/inf563/solveur/SuiteOraclesSolveur.java, lignes 209 à 215

```
@ParameterizedTest(name = "[{index}] ({0}, {1}, {2}) : {3}")
@CsvSource(delimiter = '|', textBlock = """
        1e300 | 1e-300 | 0 | c = 0 avec b² sous MIN_VALUE : tue « if (c == 0) → false »
        4     | -4     | 1 | racine double avec a ≠ 1 : tue « b / a → b * a »
        """)
@DisplayName("Cas ajoutés après l'analyse des mutants (PIT)")
void casMutation(double a, double b, double c, String mutantVise) {
```

6.7 Résultat final : 38 tués sur 39

Mesure	1er passage	Passage final
Mutants générés	41	39 (2 mutants du ternaire en moins)
Tués	37 (90 %)	**38 (97 %)**
Survivants	4	1 (M3, équivalent)
Score hors équivalents	—	**38 / (39 − 1) = 100 %**
Seuil (85 %)	**atteint**	**atteint, BUILD SUCCESS**

Pour chaque mutant tué, PIT indique le premier test qui l'a fait échouer :

Test « tueur »	Mutants tués	Commentaire
`pairwise`	21	à lui seul plus de la moitié
`constructionGeneree` (tests dynamiques)	7	surtout le cas dynamique n° 6
`parReference`	5	dont la racine double (1, −2, 1)
`discriminantNegatif`	2	
`casMutation`	2	M1 et M4, comme prévu
`echelle` (métamorphique)	1	
Total	**38**	

Attention : PIT ne retient qu'**un** test tueur par mutant (le premier qui échoue, dans l'ordre qu'il choisit) ; d'autres tests auraient souvent tué le même mutant.

6.8 Les leçons des survivants

- **Donnée trop particulière** (M4) : a = 1 cache la différence entre `/` et `*`.
- **Valeur limite manquante** (M1) : les mutants survivent quand les tests évitent les frontières (diapo 137).
- **Code inutile** (M2) : un mutant équivalent signale souvent une condition qui ne sert à rien.
- **Équivalent véritable** (M3) : on le justifie par écrit, au cas par cas (diapo 135).

6.9 Couverture et mutation : complémentaires

	Couverture (JaCoCo)	Mutation (PIT)
Question	Quel code est exécuté ?	Quelles fautes sont détectées ?
Voit l'oracle ?	Non	Oui : un test sans assertion ne tue rien
Coût	Faible (une exécution)	Élevé (une exécution par mutant)
Ici	100 % des branches dès l'étape 3	4 survivants malgré ces 100 %, puis 38/39
Limite	Rassurante à tort	Mutants équivalents à analyser

À retenir

PIT a généré 39 mutants de `SolveurStable` et en a tué 38 (97 %). Le survivant est équivalent et justifié : 100 % hors équivalent. Les 4 survivants du premier passage ont produit 2 tests (valeur limite, donnée moins particulière) et une simplification du code.

7. Exécuter, lire les rapports, dépanner

7.1 Les commandes (terminal VS Code : Ctrl + ù)

Commande	Résultat attendu
`.\mvnw.cmd test`	158 tests, 0 échec, BUILD SUCCESS
`.\mvnw.cmd test -Pnaif`	142 tests, 28 échecs : BUILD FAILURE (voulu)
`.\mvnw.cmd test "-Dsolveur.impl=inf563.solveur.SolveurStableDiapo54"`	142 tests, 5 échecs
`.\mvnw.cmd clean verify`	tests + JaCoCo : `All coverage checks have been met`
`start target\site\jacoco\index.html`	ouvre le rapport de couverture dans le navigateur
`.\mvnw.cmd test-compile org.pitest:pitest-maven:mutationCoverage`	`Generated 39 mutations Killed 38 (97%)` , BUILD SUCCESS
`start target\pit-reports\index.html`	ouvre le rapport de mutation

Le dossier `target/` n'est pas versionné par Git : une copie des deux rapports est rangée dans `doc/rapports/jacoco/` et `doc/rapports/pit/` (ouvrir `index.html` par un double-clic).

7.2 Dans VS Code, pas à pas

1. Ouvrir le dossier du projet et attendre « Java : Ready » en bas à droite.
2. Ouvrir le **Test Explorer** (icône fiole) : cinq classes apparaissent : `ChargeurSolveurTest` (11), `PairwiseTest` (5), `SolveurStableTest` (142), `SolveurNaifTest` (142, dont 28 rouges voulus) et `SolveurConfigurableTest` , affichée **désactivée** tant que `solveur.impl` n'est pas donnée : c'est normal.
3. Déplier « Pairwise : 9 catégories de réels, 81 cas au lieu de 729 » : 81 lignes, une par cas, avec les catégories de a, b et c. Chacune se relance seule avec ▶.
4. Pour la couverture et la mutation, utiliser le terminal (§ 7.1) puis ouvrir les rapports.

Pour aller plus loin : la classe configurable dans VS Code

Ajouter dans `.vscode/settings.json` : `"java.test.config": { "vmArgs": ["-Dsolveur.impl=inf563.solveur.SolveurStableDiapo54"] }` , puis lancer `SolveurConfigurableTest` depuis le Test Explorer.

7.3 Dépannage

Symptôme	Solution
PIT : `Coverage generation minion exited abnormally`	Relancer avec `"-Dverbose=true"` pour voir la cause. Si « OutputDirectoryProvider not available » : versions JUnit / `pitest-junit5-plugin` non alignées (le pom utilise 1.2.3).
PIT signale des tests qui échouent sans mutation	La suite doit être verte : ne pas activer `-Pnaif` ; vérifier `excludedGroups` .
Erreur Maven du type « Unknown lifecycle phase » avec `-Dsolveur.impl`	Sous PowerShell, mettre l'option `-D…` entre guillemets.
Tous les tests en ERROR « classe introuvable »	Donner le nom complet (avec le paquetage) d'une classe compilée dans `src/main/java` ou `src/test/java` .

Symptôme	Solution
JaCoCo : `Rule violated … branches covered ratio is …`	Une branche n'est plus couverte : ouvrir le rapport et chercher les lignes jaunes ou rouges.
Pas de rapport JaCoCo	Lancer `verify` (pas `test`) ; si des tests échouent, la phase `verify` n'est pas atteinte.
`java` introuvable ou mauvaise version	`JAVA_HOME` doit pointer vers le JDK 21 (`C:\Program Files\Eclipse Adoptium\jdk-21.0.13.9-hotspot`).

8. Questions probables de l'enseignant et réponses

> Réponses formulées comme à l'oral : courtes, précises, avec un chiffre du projet quand c'est possible. Entre parenthèses, la section où le sujet est approfondi.

8.1 La réflexion

Q1. Pourquoi utiliser la réflexion ici ?

Pour que les tests ne dépendent d'aucune classe concrète : le nom de l'implémentation est une donnée, choisie au lancement. On teste une autre implémentation sans modifier ni recompiler les tests, comme le demande la consigne. (§ 2.1)

Q2. Que fait `Class.forName("inf563.solveur.SolveurStable")` ?

Il demande au chargeur de classes de trouver, charger et initialiser la classe de ce nom complet, et renvoie l'objet `Class` qui la décrit. Si elle n'est pas sur le classpath : `ClassNotFoundException`. (§ 2.4)

Q3. Pourquoi une interface ?

Il faut un type connu à la compilation pour appeler `racines()` sur un objet dont la classe n'est connue qu'à l'exécution. L'interface porte le prototype et le contrat. (§ 2.3)

Q4. Pourquoi ne pas écrire simplement `new SolveurStable()` ?

Ce serait une dépendance directe à la classe concrète, exactement ce que la consigne interdit : changer d'implémentation obligerait à modifier le test. (§ 2.1)

Q5. Que se passe-t-il si la classe n'a pas de constructeur public sans argument ?

`getConstructor()` lève `NoSuchMethodException`, transformée en `IllegalArgumentException` « … n'a pas de constructeur public sans argument » ; chaque test se termine en ERROR. C'est testé avec `ChargeurSolveurTest$SansConstructeurVide`. (§ 2.9)

Q6. Et si le constructeur lui-même lève une exception ?

`newInstance()` l'enveloppe dans une `InvocationTargetException` ; on récupère la vraie cause par `getCause()` et on produit « le constructeur de … a levé une exception ». (§ 2.4)

Q7. JUnit utilise-t-il aussi la réflexion ?

Oui : il découvre les méthodes annotées, crée l'instance de la classe de test par son constructeur et invoque les méthodes par réflexion. Notre chargeur fait la même chose pour le système sous test. (§ 2.2)

Q8. Pourquoi un nom faux donne-t-il ERROR et pas FAIL ?

FAIL, c'est un oracle (une assertion) qui rejette une sortie ; ici aucune sortie n'est produite : une exception imprévue est levée pendant la construction de la classe de test. (§ 2.8)

8.2 Le pairwise et l'oracle

Q9. Quelles sont vos neuf catégories ?

Les classes usuelles des réels de la diapo 93 : négatif, positif, zéro signé, très petit, très grand, valeur spéciale (NaN), non représentable (0,1), entier au-delà de 2^{53}, valeur entière ; un représentant chacune. (§ 3.1)

Q10. Pourquoi 81 cas ?

Pour couvrir toutes les paires (a, b), il faut au moins 9 × 9 = 81 cas ; le carré latin atteint ce minimum tout en couvrant aussi (a, c) et (b, c). L'exhaustif en demanderait 729. (§ 3.3)

Q11. Prouvez que chaque paire est couverte.

Cas (i, j) = (V[i], V[j], V[(i + j) mod 9]). (a, b) : tous les couples (i, j). (a, c) : à i fixé, (i + j) mod 9 parcourt les 9 valeurs quand j varie. (b, c) : idem à j fixé. `PairwiseTest` le vérifie : 81 paires distinctes pour chaque couple, chacune une fois. (§ 3.3, 3.4)

Q12. Pourquoi un carré latin plutôt que PICT ?

Trois paramètres de même taille : le carré latin donne l'optimum (81), démontrable, sans outil externe. PICT est utile avec beaucoup de paramètres, des tailles différentes ou des contraintes ; il ne ferait pas mieux que 81 ici. (§ 3.3)

Q13. Pourquoi garder NaN comme catégorie s'il est invalide ?

La diapo 93 le range parmi les valeurs à tester, et un test robuste couvre aussi les classes invalides (diapo 91). Pour ces cas, l'oracle attend une `IllegalArgumentException` (règle ajoutée au contrat). (§ 3.1)

Q14. Comment respectez-vous « une seule classe invalide par cas » ?

Le pairwise combine parfois deux classes invalides ; il ne vérifie alors que la levée de l'exception. Le test `classeInvalide` reprend chaque valeur invalide seule (NaN, ±∞ sur a, b, c ; a = ±0) avec des valeurs valides ailleurs : 11 cas. (§ 3.7)

Q15. Le pairwise a-t-il raté des défauts par rapport à l'exhaustif ?

Pas d'implémentation défectueuse ratée : il détecte la version de la diapo 54 (4/81) et la naïve (15/81), dans la même proportion que l'exhaustif (33/729 et 145/729) ; la version corrigée passe les 729. (§ 3.8)

Q16. Comment l'oracle des cas générés obtient-il le résultat attendu ?

`ReferenceExacte` : Δ exact en `BigDecimal` (nombre de racines sûr), racines par la formule stable à 40 chiffres arrondies au double le plus proche, Tol = 10·κ·ε pour chaque racine. (§ 3.6)

Q17. Votre `max(|RA|, MIN_NORMAL)`, n'est-ce pas le piège de la diapo 70 ?

Non : `max(1, |RA|)` rend la tolérance absolue pour toute racine < 1. MIN_NORMAL ≈ $2{,}2 \cdot 10^{-308}$ n'agit que chez les sous-normaux, où le double n'a plus de précision relative ; ailleurs le critère reste relatif. (§ 3.6)

Q18. Et une racine plus grande que MAX_VALUE ?

La référence arrondie vaut ±∞ ; on exige alors y = ±∞ exactement, avec le même signe. (§ 3.6)

8.3 La couverture (JaCoCo)

Q19. Comment JaCoCo mesure-t-il la couverture ?

Un agent Java instrumente le bytecode au chargement des classes ; les compteurs sont écrits dans `target/jacoco.exec`, puis `report` produit le rapport et `check` applique le seuil, en phase `verify`. (§ 5.1)

Q20. Pourquoi exclure `SolveurNaif` et `Demo` ?

On évalue l'implémentation livrée. `Demo` n'est qu'un affichage ; `SolveurNaif` est un contre-exemple dont la suite est exclue du build par défaut : il apparaîtrait non couvert et fausserait la mesure. (§ 5.2)

Q21. Quelle est la complexité cyclomatique de `racines` ?

V(G) = 9 : 8 décisions (3 `isFinite`, a = 0, c = 0, b = 0, d < 0, d = 0) + 1, ou E − N + 2 = 24 − 17 + 2 ; JaCoCo affiche bien 9. Il faut donc au moins 9 tests pour une base de chemins : c'est le tableau du § 5.6. (§ 5.5)

Q22. 100 % de couverture : votre suite est donc parfaite ?

Non : la couverture mesure ce qui est exécuté, pas ce qui est vérifié (diapo 132). À 100 % de branches, PIT trouvait encore 4 mutants survivants. (§ 5.7)

Q23. Que montre votre expérience à 66 % ?

Sans le pairwise ni les classes invalides, 12 branches sur 18 seulement sont couvertes et le build échoue sur le seuil : ce sont ces nouveaux tests qui couvrent les cas « coefficient non fini » et « c = 0 ». (§ 5.4)

8.4 La mutation (PIT)

Q24. Qu'est-ce qu'un mutant ? Un mutant tué ?

Une copie du code avec une petite faute injectée (par exemple `/` → `*`). Il est tué si au moins un test échoue sur lui ; sinon il survit. (§ 6.1)

Q25. Qu'est-ce qu'un mutant équivalent ? Un exemple du projet ?

Un mutant syntaxiquement différent mais de même comportement : aucun test ne peut le tuer. Ici `e += e & 1` → `e -= e & 1` : arrondir l'exposant au pair inférieur ou supérieur donne la même $\sqrt{\Delta}$, bit pour bit. (§ 6.6)

Q26. Quel est votre score de mutation ?

38 tués sur 39 générés, soit 97 % ; hors mutant équivalent : 38 / (39 − 1) = 100 %. Le seuil configuré est 85 %. (§ 6.7)

Q27. Pourquoi PIT exige-t-il une suite verte ?

Il exécute d'abord les tests sans mutation pour savoir lesquels couvrent chaque ligne ; un test déjà rouge « tuerait » tous les mutants et rendrait le score dénué de sens. D'où l'exclusion des tags `naif` et `configurable`. (§ 6.3)

Q28. Pourquoi avoir changé une version de la diapo 138 ?

`pitest-junit5-plugin` 1.2.1 tire `junit-platform-launcher` 1.9.2, incompatible avec JUnit 5.13 (« OutputDirectoryProvider not available ») ; la 1.2.3 corrige. `pitest-maven` reste en 1.15.3. (§ 6.4)

Q29. Qu'avez-vous appris des survivants ?

M4 : un test de racine double avec a = 1 ne distingue pas `b/a` de `b*a` (donnée trop particulière). M1 : il manquait la valeur limite c = 0 avec b^2 sous MIN_VALUE. M2 : une condition inutile, supprimée. (§ 6.6, 6.8)

Q30. Quels tests tuent le plus de mutants ?

Le pairwise : 21 des 38 (premier test tueur rapporté par PIT), puis les tests dynamiques par construction (7) et l'oracle par référence (5). Les relations métamorphiques n'en tuent qu'un. (§ 6.7)

8.5 Questions d'ouverture

Q31. Pourquoi avoir modifié la version stable du cours ?

Le pairwise a trouvé 4 entrées légales, à racines représentables, où elle renvoie 1 racine au lieu de 2 (sous-dépassement après la mise à l'échelle commune). Trouver puis corriger, c'est le TP de la séance 2 ; l'original est gardé dans `src/test` pour le montrer. (partie 4)

Q32. La couverture de lignes de PIT et celle de JaCoCo, c'est la même chose ?

Non : PIT ne calcule qu'une couverture de lignes (19/19), pour savoir quels tests lancer sur chaque mutant. JaCoCo mesure instructions, branches, lignes, complexité et méthodes. (§ 5.3, 6.5)

Q33. Que feriez-vous ensuite ?

Du 3-wise sur les catégories à risque (option `/o:3` de PICT), les sous-normaux (MIN_VALUE) comme dixième catégorie, des mutateurs plus forts (groupe STRONGER), et JaCoCo et PIT dans un pipeline d'intégration continue (diapo 312), avec l'analyse statique de la séance 10 (diapo 229).

9. Mise à jour des chiffres de l'ancien document

> Le premier document décrit le projet au stade de la séance 3. Voici ce qui y est devenu faux, et où lire la version à jour.

9.1 Les chiffres

Élément	Ancien document	Aujourd'hui
Tests par classe de suite	84	142
Tests de `mvnw test`	84	158 (142 + 11 + 5)
Échecs de la version naïve	18	28

Élément	Ancien document	Aujourd'hui
Méthodes de test de la suite	11 (1 @Test, 9 @ParameterizedTest, 1 @TestFactory)	13 (1, 11, 1)
Fichiers Java	5 (2 main + 3 test)	15 (4 main + 11 test)
Implémentations	`Solveur.racines`, `Solveur.racinesNaif`	`SolveurStable`, `SolveurNaif` (+ `SolveurStableDiapo54` en test)
Bloc combinatoire	`combinaisons`, 36 cas, 6 échecs naïfs	`pairwise`, 81 cas, 15 échecs naïfs
Contrat	a = 0 → exception	+ coefficient NaN ou ±∞ → exception

9.2 Les sections à relire ici

Ancien document	Ce qui a changé	Lire ici
3. Architecture, arborescence, diagramme de classes	Interface + deux classes ; chargeur par réflexion ; 5 classes de test	§ 1.3, partie 2
5.5 `@MethodSource` et les combinaisons	`@MethodSource` fournit désormais les 81 cas pairwise	§ 3.5
6. `Solveur.java` ligne par ligne	`SolveurNaif` = ancien `naif()` + règle NaN ; `SolveurStable` réécrite	partie 4
7.6 Bloc 4 : toutes les combinaisons	Remplacé par le pairwise	partie 3
8.5 à 8.7 Échecs naïfs, « détecte / ne détecte pas », camarade	28 échecs (tableau ci-dessous) ; camarade par `-Dsolveur.impl`	§ 9.3, § 2.10
9.7 Commandes	`verify` (JaCoCo), PIT, `-Dsolveur.impl`	§ 7.1
10. Bilan et suites possibles	Les suites « chapitres IV et V » sont réalisées	parties 3 à 6
11. Questions de l'enseignant	Q1, Q3, Q4, Q5, Q8 citent l'ancienne architecture	partie 8
Annexes (code source)	Code actuel	annexes

9.3 « Détecte / ne détecte pas » : la version naïve, bloc par bloc

Bloc de tests	Échecs naïfs	Bloc de tests	Échecs naïfs
`parReference`	4 / 8	`construction`	1 / 3
`aNul`	0 / 1	`pairwise`	15 / 81
`discriminantNegatif`	0 / 3	`casMutation`	1 / 2
`residu`	1 / 3	`classeInvalide`	0 / 11
`viete`	1 / 2	`constructionGeneree`	4 / 20
`echelle`	0 / 3	`differentiel`	1 / 3
`oppose`	0 / 2	**Total**	**28 / 142**

Les métamorphiques et le contrat ne voient toujours pas la version naïve (fausse mais cohérente) ; le pairwise apporte à lui seul 15 détections nouvelles.

Annexes : le code source actuel

> Code tel qu'il est dans le dépôt le 4 novembre 2026 (copié automatiquement depuis les fichiers lors de la génération de ce document).

A. src/main/java/inf563/solveur/SolveurSecondDegre.java

```java
package inf563.solveur;

/**
 * Fil rouge A : contrat d'un solveur de l'équation du second degré a·x² + b·x + c = 0.
 *
 * <p>Contrat commun à toutes les implémentations :
 * <ul>
 *   <li>a, b ou c vaut NaN, +∞ ou −∞ : IllegalArgumentException (coefficient non fini) ;</li>
 *   <li>a = 0 : IllegalArgumentException (ce n'est plus une équation du second degré) ;</li>
 *   <li>Δ &lt; 0 : tableau vide (pas de racine réelle) ;</li>
 *   <li>Δ = 0 : une racine double ;</li>
 *   <li>Δ &gt; 0 : deux racines triées dans l'ordre croissant.</li>
 * </ul>
 *
 * <p>Les tests ne dépendent que de cette interface : l'implémentation à tester est chargée
 * par réflexion à partir de son nom (voir ChargeurSolveur dans src/test). Une implémentation
 * doit donc être une classe publique avec un constructeur public sans argument.
 */
public interface SolveurSecondDegre {

    /** Racines réelles triées de a·x² + b·x + c = 0. */
    double[] racines(double a, double b, double c);
}
```

B. src/main/java/inf563/solveur/SolveurNaif.java

```java
package inf563.solveur;

/**
 * Version naïve : formule du lycée (-b ± √Δ) / 2a (diapo 51).
 * Mathématiquement juste, numériquement fragile : élimination catastrophique quand b² ≫ 4ac,
 * débordement de b² en +∞, ∞ − ∞ = NaN, absorption.
 * Sert à montrer ce que les oracles détectent.
 */
public class SolveurNaif implements SolveurSecondDegre {

    @Override
    public double[] racines(double a, double b, double c) {
        if (!Double.isFinite(a) || !Double.isFinite(b) || !Double.isFinite(c))
            throw new IllegalArgumentException("coefficient non fini (NaN ou infini)");
        if (a == 0) throw new IllegalArgumentException("a = 0");
        double d = b * b - 4 * a * c;
        if (d < 0) return new double[0];
        if (d == 0) return new double[]{-b / (2 * a)};
        double r1 = (-b - Math.sqrt(d)) / (2 * a);
        double r2 = (-b + Math.sqrt(d)) / (2 * a);
        return new double[]{Math.min(r1, r2), Math.max(r1, r2)};
    }
}
```

C. src/main/java/inf563/solveur/SolveurStable.java

```java
package inf563.solveur;

/**
 * Version stable : formule stable + mise à l'échelle par puissances de 2 (diapo 54), corrigée.
 *
 * <p>q additionne deux termes de même signe (plus de soustraction de nombres proches),
 * la seconde racine vient de Viète (r1·r2 = c/a). Multiplier par 2^k est exact en binaire.
 *
 * <p>Correction trouvée grâce au test pairwise : la diapo 54 divise les trois coefficients par la même
 * puissance de 2 ; les petits coefficients passent alors sous MIN_VALUE et Δ devient 0 à tort
 * (ex. (1e-300, 2.5, 1e300) : 1 racine au lieu de 2). Ici on écrit chaque coefficient x = m·2^e
 * (mantisse m dans [1, 2[) et on calcule Δ = b² − 4ac = 2^e·d en mettant à l'échelle b² et 4ac
```

```java
 * SÉPARÉMENT : ni l'un ni l'autre ne peut déborder (+∞) ou s'annuler (sous MIN_VALUE).
 * Les racines sont ensuite obtenues par une seule division chacune (q/a et c/q) : si la vraie racine
 * dépasse MAX_VALUE ou passe sous MIN_VALUE, l'arrondi donne ±∞ ou 0, comme il se doit.
 */
public class SolveurStable implements SolveurSecondDegre {

    @Override
    public double[] racines(double a, double b, double c) {
        if (!Double.isFinite(a) || !Double.isFinite(b) || !Double.isFinite(c))
            throw new IllegalArgumentException("coefficient non fini (NaN ou infini)");
        if (a == 0) throw new IllegalArgumentException("a = 0");
        if (c == 0) {                                   // x·(a·x + b) = 0 : racines 0 et −b/a
            if (b == 0) return new double[]{0.0};       // Δ = 0 : racine double 0
            double r = -b / a;
            return new double[]{Math.min(r, 0.0), Math.max(r, 0.0)};
        }
        // x = m·2^e, mantisse m dans [1, 2[ : multiplier par une puissance de 2 est exact
        int ea = Math.getExponent(a), eb = Math.getExponent(b), ec = Math.getExponent(c);
        double ma = Math.scalb(a, -ea), mb = Math.scalb(b, -eb), mc = Math.scalb(c, -ec);
        // Δ = 2^e·d : e = exposant du plus grand des deux termes b² et 4ac, rendu pair pour la racine carrée
        // (b = 0 : getExponent(0) = −1023, donc 2·eb ≤ ea + ec et le max choisit bien 4ac)
        int e = Math.max(2 * eb, ea + ec);
        e += e & 1;                                     // pair supérieur (le pair inférieur marcherait aussi)
        double d = Math.scalb(mb * mb, 2 * eb - e) - Math.scalb(4 * ma * mc, ea + ec - e);
        if (d < 0) return new double[0];
        if (d == 0) return new double[]{-(b / a) / 2};
        double s = Math.scalb(Math.sqrt(d), e / 2);     // √Δ = √d · 2^(e/2)
        double q = -0.5 * (b + Math.copySign(s, b));
        double r1 = q / a, r2 = c / q;                  // Viète : r1 * r2 = c / a
        return new double[]{Math.min(r1, r2), Math.max(r1, r2)};
    }
}
```

D. src/test/java/inf563/solveur/ChargeurSolveur.java

```java
package inf563.solveur;

import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Modifier;

/**
 * Chargement et instanciation de l'implémentation à tester par réflexion (Java Reflection API).
 *
 * <p>Les tests ne connaissent que l'interface {@link SolveurSecondDegre} et un NOM de classe
 * (une chaîne) : aucune classe concrète n'est importée ni instanciée avec {@code new}.
 * On peut donc tester une autre implémentation sans recompiler ni modifier les tests.
 */
final class ChargeurSolveur {

    /** Propriété système qui choisit l'implémentation au lancement : -Dsolveur.impl=nom.complet.DeLaClasse */
    static final String PROPRIETE = "solveur.impl";

    private ChargeurSolveur() {
    }

    /**
     * @param nomClasse nom complet de la classe, par ex. "inf563.solveur.SolveurStable"
     * @return une nouvelle instance de cette classe, vue uniquement à travers l'interface
     * @throws IllegalArgumentException si la classe est introuvable ou inutilisable (message explicite)
     */
    static SolveurSecondDegre charger(String nomClasse) {
        if (nomClasse == null || nomClasse.isBlank()) {
            throw new IllegalArgumentException("aucun nom de classe fourni (propriété -D" + PROPRIETE + " ?)");
        }
        // 1. Chargement : le nom (texte) devient un objet Class, sans dépendance à la compilation
        Class<?> classe;
        try {
            classe = Class.forName(nomClasse.strip());
        } catch (ClassNotFoundException e) {
            throw new IllegalArgumentException("classe introuvable : " + nomClasse, e);
        }
        // 2. Vérification du contrat : la classe doit implémenter l'interface et être instanciable
        if (!SolveurSecondDegre.class.isAssignableFrom(classe)) {
            throw new IllegalArgumentException(classe.getName() + " n'implémente pas " + SolveurSecondDegre.class.getName());
        }
        if (classe.isInterface() || Modifier.isAbstract(classe.getModifiers())) {
            throw new IllegalArgumentException(classe.getName() + " est une interface ou une classe abstraite");
        }
        // 3. Instanciation par le constructeur public sans argument
        try {
```

```
            return classe.asSubclass(SolveurSecondDegre.class).getConstructor().newInstance();
        } catch (NoSuchMethodException e) {
            throw new IllegalArgumentException(classe.getName() + " n'a pas de constructeur public sans argument", e);
        } catch (InvocationTargetException e) {
            throw new IllegalArgumentException("le constructeur de " + classe.getName() + " a levé une exception",
e.getCause());
        } catch (ReflectiveOperationException e) {     // InstantiationException, IllegalAccessException
            throw new IllegalArgumentException("impossible d'instancier " + classe.getName(), e);
        }
    }
}
```

E. src/test/java/inf563/solveur/CategorieReel.java

```
package inf563.solveur;

/**
 * Les neuf catégories de valeurs d'un réel (double), d'après la diapo 93 (« classes usuelles : réels »),
 * avec un représentant par catégorie. Chaque coefficient a, b, c prend ses valeurs dans ces catégories.
 *
 * <p>Classes valides et invalides : NaN est invalide pour a, b et c ; zéro est invalide pour a seulement
 * (a = 0 : ce n'est plus une équation du second degré). Les autres sont valides partout.
 */
enum CategorieReel {

    NEGATIF("négatif", -3.7),
    POSITIF("positif", 2.5),
    ZERO_SIGNE("zéro signé", -0.0),                        // le piège : -0.0 == 0.0 est vrai
    TRES_PETIT("très petit", 1e-300),
    TRES_GRAND("très grand", 1e300),
    SPECIALE("valeur spéciale", Double.NaN),               // ±∞ : voir le test « classes invalides »
    NON_REPRESENTABLE("non représentable", 0.1),           // 0.1 n'existe pas en binaire
    ENTIER_AU_DELA_2_53("entier > 2^53", (double) 9_007_199_254_740_993L), // devient 2^53
    VALEUR_ENTIERE("valeur entière", 3.0);

    final String libelle;
    final double valeur;

    CategorieReel(String libelle, double valeur) {
        this.libelle = libelle;
        this.valeur = valeur;
    }

    @Override
    public String toString() {
        return libelle + " (" + valeur + ")";
    }
}
```

F. src/test/java/inf563/solveur/Pairwise.java

```
package inf563.solveur;

import java.util.ArrayList;
import java.util.List;

/**
 * Stratégie pairwise (2-wise, diapo 109) pour 3 paramètres (a, b, c) à 9 valeurs chacun.
 *
 * <p>Exhaustif : 9³ = 729 cas. Pairwise : chaque PAIRE de valeurs de deux paramètres apparaît au moins
 * une fois. Il faut au moins 9 × 9 = 81 cas (toutes les paires (a, b)), et 81 suffisent : c'est un
 * tableau orthogonal L81(9³), construit par un carré Latin, comme les tableaux L4 et L9 des diapos 109-110.
 *
 * <p>Construction : cas (i, j) = (V[i], V[j], V[(i + j) mod 9]) pour i, j = 0..8.
 * <ul>
 *   <li>(a, b) : tous les couples (i, j) sont parcourus ;</li>
 *   <li>(a, c) : à i fixé, (i + j) mod 9 prend les 9 valeurs quand j varie ;</li>
 *   <li>(b, c) : à j fixé, (i + j) mod 9 prend les 9 valeurs quand i varie.</li>
 * </ul>
 * Chaque paire apparaît donc exactement une fois (vérifié par PairwiseTest).
 */
final class Pairwise {

    private Pairwise() {
    }

    /** Les 81 cas pairwise : chaque cas est un triplet {catégorie de a, catégorie de b, catégorie de c}. */
    static List<CategorieReel[]> tableauOrthogonal() {
```

```
        CategorieReel[] v = CategorieReel.values();
        int n = v.length;
        List<CategorieReel[]> cas = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                cas.add(new CategorieReel[]{v[i], v[j], v[(i + j) % n]});
            }
        }
        return cas;
    }

    /** Les 729 cas exhaustifs (produit cartésien), pour comparer. */
    static List<CategorieReel[]> exhaustif() {
        List<CategorieReel[]> cas = new ArrayList<>();
        for (CategorieReel a : CategorieReel.values()) {
            for (CategorieReel b : CategorieReel.values()) {
                for (CategorieReel c : CategorieReel.values()) {
                    cas.add(new CategorieReel[]{a, b, c});
                }
            }
        }
        return cas;
    }
}
```

G. src/test/java/inf563/solveur/PairwiseTest.java

```
package inf563.solveur;

import static org.junit.jupiter.api.Assertions.assertEquals;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

/** On teste aussi les données de test : le tableau pairwise a-t-il bien la propriété annoncée ? */
@DisplayName("Générateur pairwise : 81 cas couvrent toutes les paires")
class PairwiseTest {

    static final int N = CategorieReel.values().length;    // 9

    @Test
    @DisplayName("81 cas pairwise contre 9³ = 729 cas exhaustifs")
    void taille() {
        assertEquals(9, N);
        assertEquals(81, Pairwise.tableauOrthogonal().size());
        assertEquals(729, Pairwise.exhaustif().size());
    }

    /** Pour deux paramètres p et q (0 = a, 1 = b, 2 = c), chacune des 81 paires apparaît exactement une fois. */
    @ParameterizedTest(name = "paires ({1}, {2}) : les 81 paires, chacune une fois")
    @CsvSource({"0, a, b, 1", "0, a, c, 2", "1, b, c, 2"})
    @DisplayName("Chaque paire de valeurs de deux paramètres est couverte")
    void toutesLesPaires(int p, String nomP, String nomQ, int q) {
        Map<String, Integer> occurrences = new HashMap<>();
        for (CategorieReel[] cas : Pairwise.tableauOrthogonal()) {
            occurrences.merge(cas[p] + " / " + cas[q], 1, Integer::sum);
        }
        assertEquals(N * N, occurrences.size(), "nombre de paires distinctes");
        occurrences.forEach((paire, n) -> assertEquals(1, n, "paire " + paire));
    }

    /** Each-choice (1-wise) en découle : chaque catégorie est prise 9 fois par chaque paramètre. */
    @Test
    @DisplayName("Chaque catégorie apparaît 9 fois pour a, pour b et pour c")
    void equilibre() {
        List<CategorieReel[]> cas = Pairwise.tableauOrthogonal();
        for (int p = 0; p < 3; p++) {
            for (CategorieReel valeur : CategorieReel.values()) {
                final int param = p;
                long n = cas.stream().filter(t -> t[param] == valeur).count();
                assertEquals(N, n, "paramètre " + p + ", " + valeur);
            }
        }
```

```
    }
}
```

H. src/test/java/inf563/solveur/ReferenceExacte.java

```java
package inf563.solveur;

import java.math.BigDecimal;
import java.math.MathContext;

/**
 * Résultat attendu (RA) et tolérance (Tol) calculés pour N'IMPORTE QUELS coefficients finis, a ≠ 0.
 * Sert d'oracle au test pairwise, où les cas sont générés : on ne peut pas écrire les RA à la main.
 *
 * <ul>
 *   <li>Δ = b² − 4ac est calculé EXACTEMENT (produits BigDecimal sans arrondi) : le nombre de racines est sûr ;</li>
 *   <li>les racines viennent de la formule stable, en BigDecimal à 40 chiffres (pas d'élimination),
 *       puis sont arrondies au double le plus proche (±∞ si la vraie racine dépasse MAX_VALUE, 0 si elle est
 *       sous MIN_VALUE) ;</li>
 *   <li>Tol = 10·κ·ε pour chaque racine (RAPPORT § 2), avec κ = (|a|r² + |b||r| + |c|) / (|r|·√Δ).</li>
 * </ul>
 *
 * @param racines    RA : racines attendues, triées
 * @param tolerances Tol de chaque racine (erreur relative maximale acceptée)
 */
record ReferenceExacte(double[] racines, double[] tolerances) {

    private static final MathContext MC = new MathContext(40);
    private static final BigDecimal DEUX = BigDecimal.valueOf(2), QUATRE = BigDecimal.valueOf(4);
    private static final double EPS = Math.ulp(1.0);

    static ReferenceExacte de(double a, double b, double c) {
        BigDecimal A = new BigDecimal(a), B = new BigDecimal(b), C = new BigDecimal(c);  // valeurs exactes des double
        BigDecimal delta = B.multiply(B).subtract(QUATRE.multiply(A).multiply(C));        // exact
        if (delta.signum() < 0) {
            return new ReferenceExacte(new double[0], new double[0]);
        }
        if (delta.signum() == 0) {                    // racine double −b/2a : κ infini, on ne garde que ≈ √ε
            double r = B.negate().divide(DEUX.multiply(A), MC).doubleValue();
            return new ReferenceExacte(new double[]{r}, new double[]{10 * Math.sqrt(EPS)});
        }
        BigDecimal s = delta.sqrt(MC);
        BigDecimal q = (B.signum() >= 0 ? B.add(s) : B.subtract(s)).divide(DEUX.negate());  // −(b + signe(b)·√Δ)/2
        BigDecimal r1 = q.divide(A, MC), r2 = C.divide(q, MC);
        if (r1.compareTo(r2) > 0) {
            BigDecimal t = r1;
            r1 = r2;
            r2 = t;
        }
        return new ReferenceExacte(new double[]{r1.doubleValue(), r2.doubleValue()},
                new double[]{10 * kappa(A, B, C, r1, s) * EPS, 10 * kappa(A, B, C, r2, s) * EPS});
    }

    /** Conditionnement d'une racine simple r : κ = (|a|r² + |b||r| + |c|) / (|r|·|2ar + b|), et |2ar + b| = √Δ. */
    private static double kappa(BigDecimal a, BigDecimal b, BigDecimal c, BigDecimal r, BigDecimal racineDelta) {
        if (r.signum() == 0) {
            return 1;                                 // racine nulle (c = 0) : vérifiée en absolu, voir accepte()
        }
        BigDecimal num = a.abs().multiply(r.pow(2)).add(b.abs().multiply(r.abs())).add(c.abs());
        return num.divide(r.abs().multiply(racineDelta), MC).doubleValue();
    }

    /**
     * Verdict de l'oracle pour une racine : N = |y − RA| / |RA| ≤ Tol.
     * <ul>
     *   <li>RA = ±∞ (la vraie racine dépasse MAX_VALUE) : on exige y = RA ;</li>
     *   <li>sous MIN_NORMAL ≈ 2,2·10⁻³⁰⁸ (sous-normaux, diapo 93), le double n'a plus de précision relative :
     *       l'écart est rapporté à MIN_NORMAL au lieu de |RA|. Ce plancher est 10³⁰⁸ fois plus petit que le
     *       max(1, |RA|) de la diapo 70 : il ne masque aucune erreur sur une racine représentable.</li>
     * </ul>
     * Faux si y est NaN (toute comparaison avec NaN est fausse).
     */
    static boolean accepte(double y, double ra, double tol) {
        if (Double.isInfinite(ra)) {
            return y == ra;
        }
        return Math.abs(y - ra) <= tol * Math.max(Math.abs(ra), Double.MIN_NORMAL);
    }
}
```

I. Les trois classes de test concrètes

src/test/java/inf563/solveur/SolveurStableTest.java

```java
package inf563.solveur;

import org.junit.jupiter.api.DisplayName;

/** Toute la suite contre la version stable : tout doit être VERT. */
@DisplayName("Solveur STABLE (formule stable + mise à l'échelle)")
class SolveurStableTest extends SuiteOraclesSolveur {

    SolveurStableTest() {
        super("inf563.solveur.SolveurStable");     // un nom (texte), pas une dépendance à la classe
    }
}
```

src/test/java/inf563/solveur/SolveurNaifTest.java

```java
package inf563.solveur;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Tag;

/**
 * Toute la suite contre la version naïve : certains tests DOIVENT échouer.
 * C'est le but du TP : remplir le tableau « l'oracle détecte / ne détecte pas ».
 *
 * <p>Exclue de « mvnw test » (tag naif) ; à lancer avec « mvnw test -Pnaif »
 * ou directement depuis le Test Explorer de VS Code.
 */
@Tag("naif")
@DisplayName("Solveur NAÏF (formule du lycée)")
class SolveurNaifTest extends SuiteOraclesSolveur {

    SolveurNaifTest() {
        super("inf563.solveur.SolveurNaif");       // un nom (texte), pas une dépendance à la classe
    }
}
```

src/test/java/inf563/solveur/SolveurConfigurableTest.java

```java
package inf563.solveur;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Tag;
import org.junit.jupiter.api.condition.EnabledIfSystemProperty;

/**
 * Toute la suite contre N'IMPORTE QUELLE implémentation, choisie au lancement, sans recompiler les tests :
 * <pre>  mvnw test -Dsolveur.impl=nom.complet.DeLaClasse</pre>
 * La classe doit implémenter SolveurSecondDegre et avoir un constructeur public sans argument.
 *
 * <p>Sans la propriété, cette classe est désactivée (et exclue de « mvnw test » par le tag).
 */
@Tag("configurable")
@EnabledIfSystemProperty(named = ChargeurSolveur.PROPRIETE, matches = ".+")
@DisplayName("Solveur choisi au lancement (-Dsolveur.impl=...)")
class SolveurConfigurableTest extends SuiteOraclesSolveur {

    SolveurConfigurableTest() {
        super(System.getProperty(ChargeurSolveur.PROPRIETE));
    }
}
```

J. SuiteOraclesSolveur.java : les parties nouvelles

Le reste du fichier (blocs 1 à 3, tests dynamiques, différentiel) est inchangé depuis la séance 3 : voir l'annexe C de l'ancien document.

src/test/java/inf563/solveur/SuiteOraclesSolveur.java, lignes 41 à 52

```java
/** L'implémentation sous test, vue uniquement à travers l'interface. */
private final SolveurSecondDegre solveur;

/** JUnit crée une instance par test : chaque test reçoit une instance neuve, chargée par réflexion. */
SuiteOraclesSolveur(String nomImplementation) {
    solveur = ChargeurSolveur.charger(nomImplementation);
}

/** Le système sous test g : renvoie les racines triées de a·x² + b·x + c = 0. */
final double[] g(double a, double b, double c) {
    return solveur.racines(a, b, c);
}
```

src/test/java/inf563/solveur/SuiteOraclesSolveur.java, lignes 166 à 233

```java
// 4. Pairwise (chapitre 4) : a, b, c prennent chacun leurs valeurs dans les
//    9 catégories de réels (CategorieReel, diapo 93). Exhaustif : 9³ = 729 cas ;
//    pairwise : 81 cas, chaque paire de valeurs de deux paramètres une fois (Pairwise).
// ---------------------------------------------------------------------
static Stream<Arguments> casPairwise() {
    return Pairwise.tableauOrthogonal().stream().map(t -> Arguments.of(t[0], t[1], t[2]));
}

/**
 * Oracle de chaque cas généré :
 * <ul>
 *   <li>une classe invalide (coefficient NaN, ou a = 0) : IllegalArgumentException attendue ;</li>
 *   <li>sinon : RA et Tol calculés par ReferenceExacte (Δ exact, puis 40 chiffres), puis N ≤ Tol.</li>
 * </ul>
 */
@ParameterizedTest(name = "[{index}] a = {0} | b = {1} | c = {2}")
@MethodSource("casPairwise")
@DisplayName("Pairwise : 9 catégories de réels, 81 cas au lieu de 729")
void pairwise(CategorieReel ca, CategorieReel cb, CategorieReel cc) {
    double a = ca.valeur, b = cb.valeur, c = cc.valeur;
    if (!Double.isFinite(a) || !Double.isFinite(b) || !Double.isFinite(c) || a == 0) {
        assertThrows(IllegalArgumentException.class, () -> g(a, b, c));
        return;
    }
    verifierContreReference(a, b, c);
}

/** Oracle des cas générés : RA et Tol de ReferenceExacte, nombre de racines puis N ≤ Tol pour chacune. */
void verifierContreReference(double a, double b, double c) {
    ReferenceExacte ref = ReferenceExacte.de(a, b, c);
    double[] y = g(a, b, c);                        // y = g(DE)
    assertEquals(ref.racines().length, y.length, "nombre de racines (signe exact de Δ)");
    for (int i = 0; i < y.length; i++) {
        double ra = ref.racines()[i], tol = ref.tolerances()[i];
        assertTrue(ReferenceExacte.accepte(y[i], ra, tol),
                "racine " + i + " : y = " + y[i] + ", RA = " + ra + ", N = " + erreurRelative(y[i], ra) + ", Tol = " + tol);
    }
}

/**
 * Chapitre 5 : cas ajoutés pour tuer les mutants survivants de PIT (diapo 140, étape 4).
 * Même oracle que le pairwise. Chaque ligne indique le mutant qu'elle tue.
 */
@ParameterizedTest(name = "[{index}] ({0}, {1}, {2}) : {3}")
@CsvSource(delimiter = '|', textBlock = """
        1e300 | 1e-300 | 0 | c = 0 avec b² sous MIN_VALUE : tue « if (c == 0) → false »
        4     | -4     | 1 | racine double avec a ≠ 1 : tue « b / a → b * a »
        """)
@DisplayName("Cas ajoutés après l'analyse des mutants (PIT)")
void casMutation(double a, double b, double c, String mutantVise) {
    verifierContreReference(a, b, c);
}

/**
 * Robustesse (diapo 90) : UNE seule classe invalide par cas, des valeurs valides partout ailleurs.
 * Dans un cas pairwise qui combine deux classes invalides, la première erreur masque la seconde.
 */
@ParameterizedTest(name = "[{index}] ({0}, {1}, {2}) → IllegalArgumentException")
@CsvSource({
        "NaN, -3, 2", "Infinity, -3, 2", "-Infinity, -3, 2",
        "1, NaN, 2", "1, Infinity, 2", "1, -Infinity, 2",
        "1, -3, NaN", "1, -3, Infinity", "1, -3, -Infinity",
        "0.0, -3, 2", "-0.0, -3, 2",
})
```

```java
@DisplayName("Classes invalides : une seule par cas")
void classeInvalide(double a, double b, double c) {
    assertThrows(IllegalArgumentException.class, () -> g(a, b, c));
}
```

K. pom.xml : plugins JaCoCo et PIT

pom.xml, lignes 68 à 155

```xml
    <!-- Chapitre 5 : couverture de code avec JaCoCo (diapo 130).
         mvnw verify  → rapport dans target/site/jacoco/index.html
         prepare-agent instrumente le bytecode pendant les tests ; check fait échouer le build
         sous le seuil (quality gate). -->
    <plugin>
        <groupId>org.jacoco</groupId>
        <artifactId>jacoco-maven-plugin</artifactId>
        <version>0.8.12</version>
        <configuration>
            <!-- On mesure l'implémentation livrée (SolveurStable). Exclues :
                 Demo (affichage, pas un calcul) et SolveurNaif (contre-exemple pédagogique,
                 ses tests sont exclus du build par défaut). -->
            <excludes>
                <exclude>inf563/solveur/Demo.class</exclude>
                <exclude>inf563/solveur/SolveurNaif.class</exclude>
            </excludes>
        </configuration>
        <executions>
            <execution>
                <goals>
                    <goal>prepare-agent</goal>
                </goals>
            </execution>
            <execution>
                <id>rapport</id>
                <phase>verify</phase>
                <goals>
                    <goal>report</goal>
                    <goal>check</goal>
                </goals>
                <configuration>
                    <!-- Diapo 130 : 0.80 ; TP diapo 140 : 100 % des branches. Atteint
                         (16/16 branches de SolveurStable) : le seuil empêche de régresser. -->
                    <rules>
                        <rule>
                            <element>BUNDLE</element>
                            <limits>
                                <limit>
                                    <counter>BRANCH</counter>
                                    <value>COVEREDRATIO</value>
                                    <minimum>1.00</minimum>
                                </limit>
                            </limits>
                        </rule>
                    </rules>
                </configuration>
            </execution>
        </executions>
    </plugin>
    <!-- Chapitre 5 : test de mutation avec PIT (diapo 138).
         mvnw test-compile org.pitest:pitest-maven:mutationCoverage
         → rapport dans target/pit-reports/index.html (mutants tués / survivants, ligne par ligne) -->
    <plugin>
        <groupId>org.pitest</groupId>
        <artifactId>pitest-maven</artifactId>
        <version>1.15.3</version>
        <dependencies>
            <!-- Diapo 138 : 1.2.1. Celle-ci tire junit-platform-launcher 1.9.2, incompatible
                 avec JUnit 5.13 (« OutputDirectoryProvider not available ») : 1.2.3 corrige. -->
            <dependency>
                <groupId>org.pitest</groupId>
                <artifactId>pitest-junit5-plugin</artifactId>
                <version>1.2.3</version>
            </dependency>
        </dependencies>
        <configuration>
            <!-- on mute l'implémentation livrée, comme pour JaCoCo -->
            <targetClasses>
                <param>inf563.solveur.SolveurStable</param>
            </targetClasses>
            <!-- PIT exige une suite VERTE avant de muter : on écarte la suite naïve
```

```
                (échecs voulus) et la suite configurable (sans objet ici) -->
            <excludedGroups>
                <param>naif</param>
                <param>configurable</param>
            </excludedGroups>
            <mutators>
                <mutator>DEFAULTS</mutator>
            </mutators>
            <mutationThreshold>85</mutationThreshold>
            <timestampedReports>false</timestampedReports>
            <outputFormats>
                <param>HTML</param>
                <param>XML</param>
            </outputFormats>
        </configuration>
    </plugin>
</plugins>
```